Gas Turbine CO and NOx Emission Data set

Gas Turbine CO and NOx Emission dataset은 터키 북서부 지역에 위치한 가스 터빈으로부터 나오는 가스 배출량에 대한 데이터이다. 36733개의 행과 11개의 열로 이루어져 있으며, 한 행에는 한시간동안 측정되어 집계된 정보들이 담겨있다. 변수 설명 및 데이터의 첫 3행은 표 1, 2와 같다.

변수명	자료형	변수 설명
AT	numeric	주변 온도(Ambient temperature)
AP	numeric	주변 기압(Ambient pressure)
AH	numeric	주변 습도(Ambient humidity)
AFDP	numeric	공기 필터의 차압(Air filter difference pressure)
GTEP	numeric	가스 터빈 배압(Gas turbine exhaust pressure)
TIT	numeric	터빈 입구 온도(Turbine inlet temperature)
TAT	numeric	터빈 출구 온도(Turbine after temperature)
CDP	numeric	압축기 배출 압력(Compressor discharge pressure)
TEY	numeric	터빈 에너지 생산량(Turbine energy yield)
CO	numeric	Carbon monoxide 배출량
NOX (Target)	numeric	Nitrogen oxides 배출량(NO + NO2)

표 1 Gas Turbine emission dataset variable description

AT	AP	AH	…	TEY	CDP	CO	NOX
4.588	1018.7	83.675	…	134.67	11.898	0.327	81.952
4.293	1018.3	84.235	…	134.67	11.892	0.448	82.377
3.905	1018.4	84.858	…	135.10	12.042	0.451	83.776
…	…	…	…	…	…	…	…

표 2 Gas Turbine emission dataset 예시

11개의 변수 중 일산화 질소(NO)와 이산화 질소(NO2)의 배출량을 뜻하는 NOX를 예측변수로 두고, 나머지 10개 변수를 이용하여 이를 예측하고자 한다. 밑의 표는 NOX의 통계량을 나타내고, 그래프는 NOX 분포에 대한 히스토그램이다. 정규분포의 형태를 띄고 있으며 왼쪽 보다 오른쪽 꼬리가 좀 더 긴 분포임을 알 수 있다.

NOX (Target Variable)						
mean	std	min	25%	50%	75%	max
65.293	11.678	25.905	57.162	63.849	71.548	119.91

표 3 Target Variable NOX의 통계량

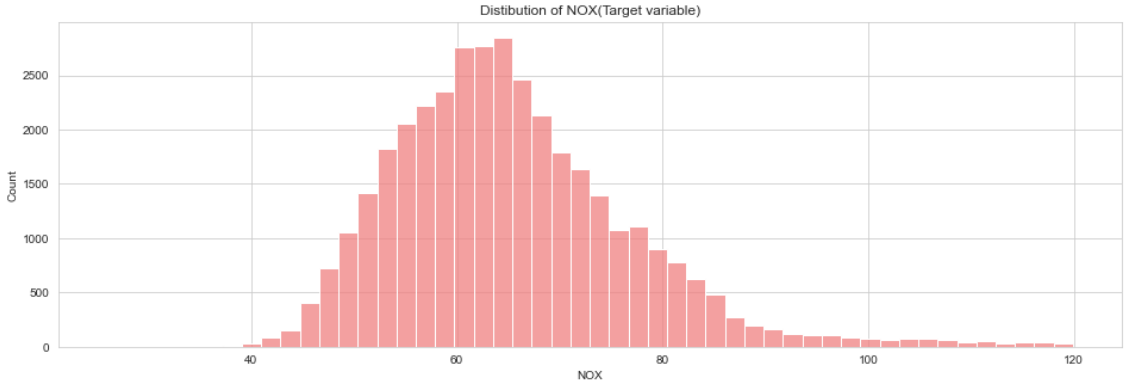


그림 1 Target variable NOX의 histogram

다음으로 설명변수로 사용할 변수들의 분포를 확인해본다. 다음 두 페이지의 그림은 각 변수의 분포에 대한 히스토그램과 변수별 예측변수와의 scatter plot 및 변수들간의 correlation plot이다.

첫번째 페이지의 변수별 히스토그램을 보면, AT와 AP 변수의 경우 분포의 중심에 위치한 데이터가 가장 많은 형태를 띄고 있으며, TIT, TAT, CO 변수의 경우 한쪽으로 크게 치우친 형태를 보인다. GTEP와 CDP의 분포가 매우 비슷한 흐름을 보이는 것 또한 알 수 있다.

두번째 페이지의 예측변수와의 scatter plot을 보면 모든 설명변수가 예측변수인 NOX와 뚜렷한 선형 관계에 있지 않은 것을 알 수 있다. Correlation plot에서 NOX 변수와 다른 변수들간의 correlation 또한 모두 색이 진하지 않아 설명변수들과의 상관관계가 높지 않다.

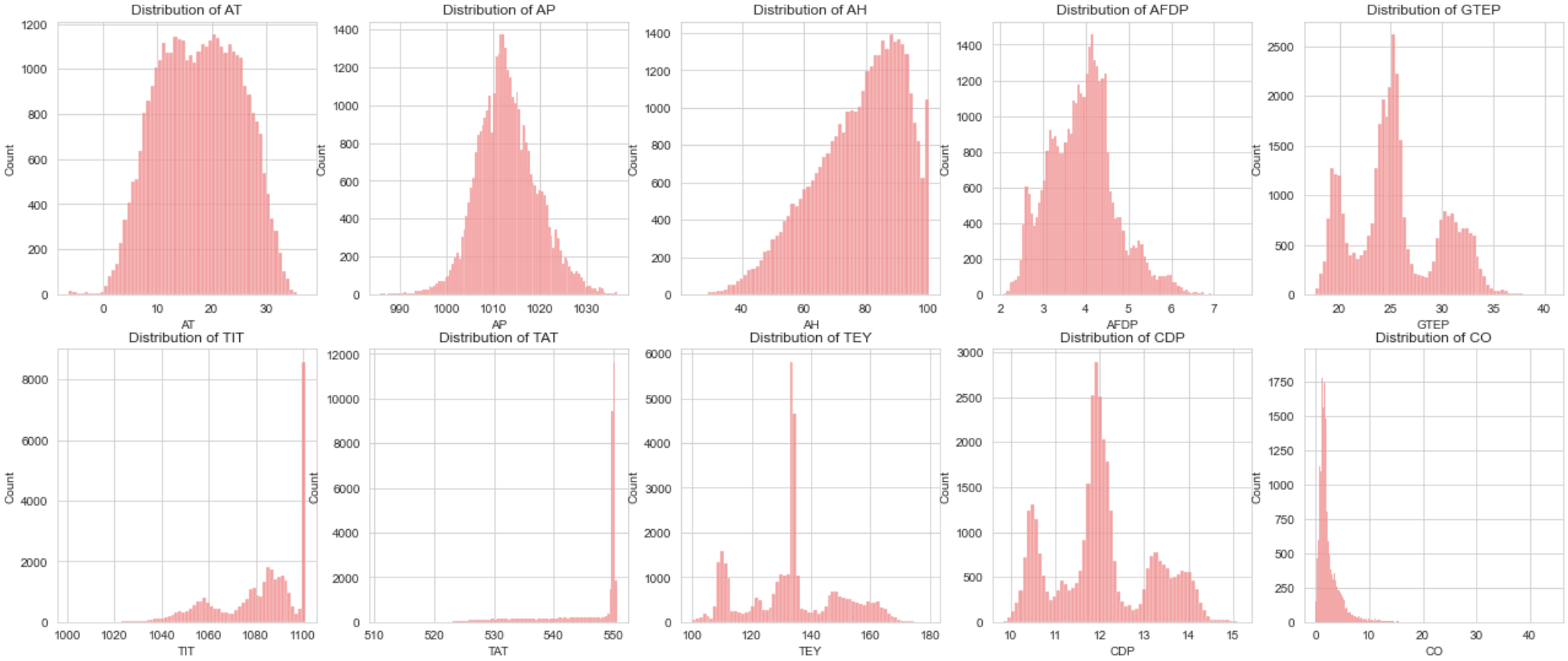


그림 2 Gas Turbine emission dataset 설명변수로 사용될 변수들의 변수별 histogram

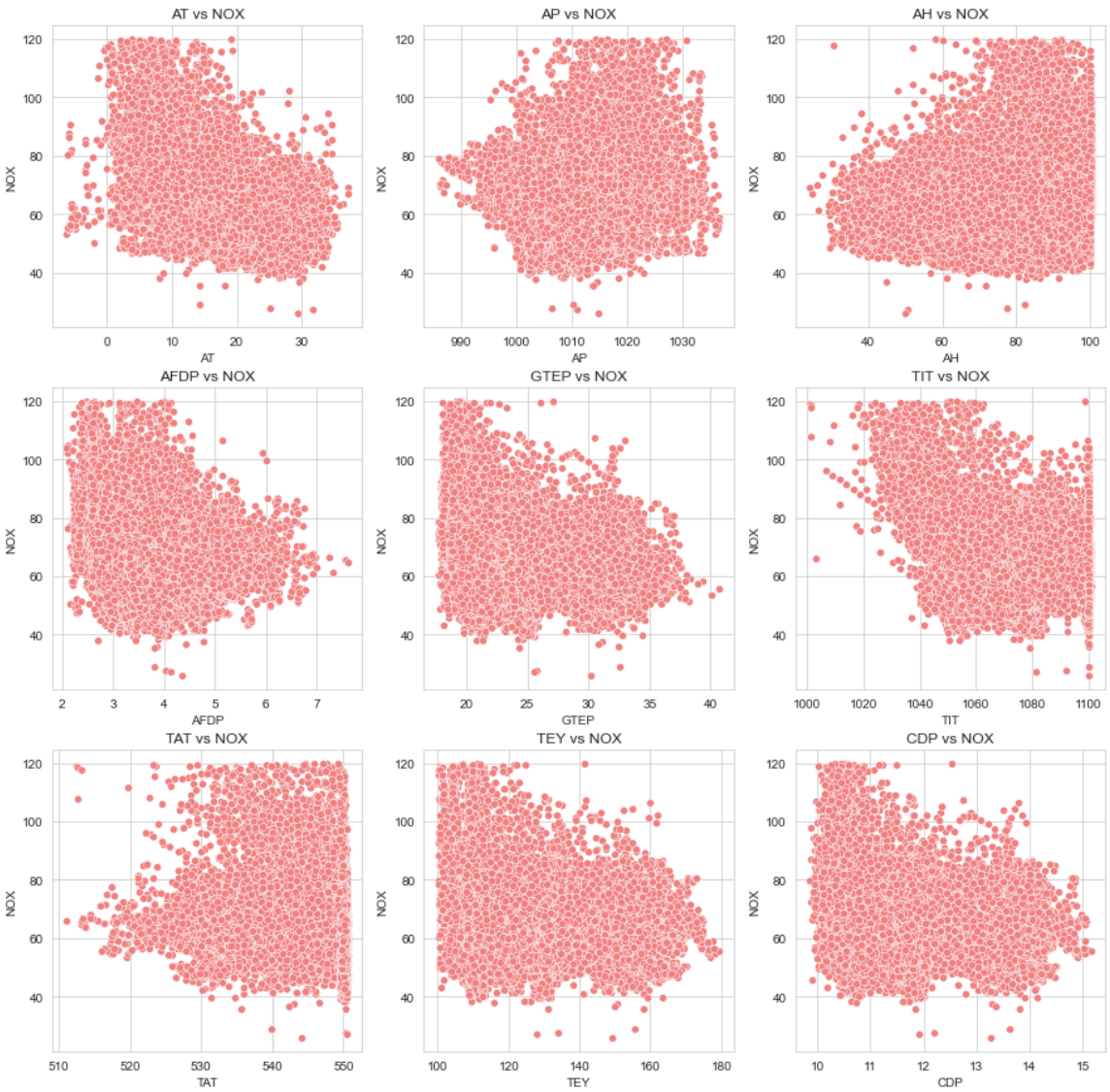


그림 3 변수별 예측변수와의 scatter plot

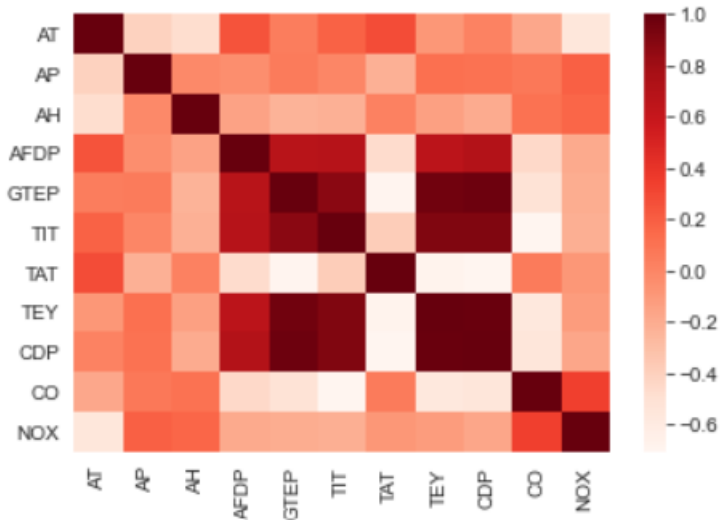


그림 4 Gas Turbine emission data correlation plot

다른 변수들은 가스 터빈의 상태 및 환경과 관련된 변수인 반면, CO 변수의 경우 CO의 배출량을 뜻하는 변수이다. CO와 NOX 변수의 상관관계에 대한 그래프는 밑의 그림과 같고, 상관관계는 0.341정도로 높은 편은 아니지만 의미상 타겟변수와 비슷한 종류의 변수이므로 CO 변수는 향후 분석에서 제외한다.

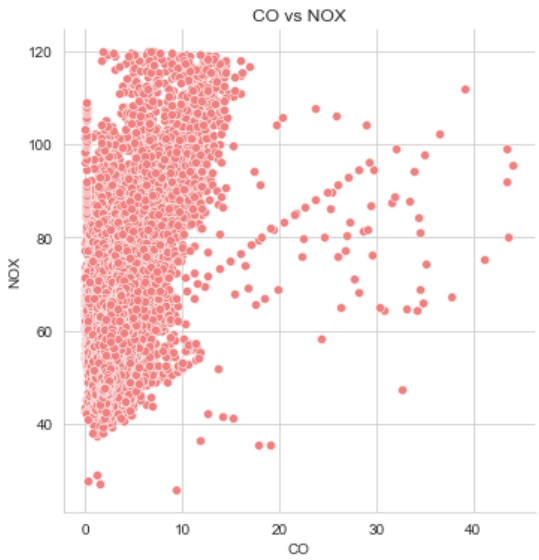


그림 5 CO 변수와 NOX 변수에 대한 scatter plot

전체 데이터 중 80%는 train set으로, 20%는 test set으로 사용하여, train set으로 모델을 적합하고 test set의 NOX 변수를 예측한다. 이후 예측 결과에 대한 평가지표를 계산하여 모델간 성능을 비교해본다.

XGBoost Implementation with R and Python

XGBoost를 이용한 NOX 배출량의 예측을 R과 Python 각각 3가지 케이스로 나눠 진행한다.

Case 1에서는 별도의 하이퍼파라미터 튜닝을 거치지 않은 XGBoost 기본 파라미터를 그대로 사용하였고, case 2에서는 case1의 default parameter와 비교했을 때 각 트리의 depth는 작게, 생성할 트리 개수는 크게, learning rate는 작게 하여 모델을 적합하였다. Case 3에서는 grid search를 통해 몇 가지 파라미터에 대한 후보군을 두어 가장 좋은 성능을 내는 파라미터 조합을 찾고, 이를 이용하여 모델을 적합하였다.

각 case에서 사용한 파라미터와 case 3의 optimal value에 대한 결과는 다음 표와 같다. Grid search 결과, 모든 파라미터가 R과 Python에서 같은 optimal value를 가졌다.

case	parameter (alias = R version name)	values	Optimal value	
			R	Python
case1 (default parameter)	max_depth	6		
	learning_rate(alias=eta)	0.3		
	n_estimator(alias=nrounds)	100		
case2	max_depth	2		
	learning_rate(alias=eta)	0.05		
	n_estimator(alias=nrounds)	1000		
case3 (girdsearch)	max_depth	[4, 6, 8]	8	8
	learning_rate(alias=eta)	[0.01, 0.1]	0.1	0.1
	n_estimators(alias=nrounds)	[100, 200]	200	200
	colsample_bytree	[0.6, 0.8]	0.8	0.8
	gamma	[0.5, 0.75, 1]	0.5	0.5

표 4 Gas turbine emission xgboost prediction 사용 파라미터

세 case 각각에 대해 모델을 실행하고 test set을 예측한 후 계산한 RMSE와 Pseudo-R^2 및 실행시간은 밑의 표와 같다. 먼저 case별로 비교해보면, 3개 케이스 중 grid search를 통해 최적의 파라미터를 찾은 case 3이 RMSE가 가장 낮고 R^2는 가장 큰 값으로 좋은 성능을 보였다. R과 Python 결과를 비교해보면, 같은 파라미터를 사용한 case 1과 case2의 RMSE 및 R^2 은 같은 값을 얻었다. Grid search를 통해 같은 optimal value를 찾은 case 3의 경우 또한 RMSE와 R^2가 약간의 차이는 있으나 크게 다르지 않은 값을 얻었다. 따라서 전체 실행 결과 중 Python으로 돌린 case 3의 모델이 가장 좋은 성능을 보였다고 판단하였다. 실행시간의 경우 전체적으로 Python이 R보다 조금 더 빨랐다.

case	Tool	RMSE	Pseudo-R^2	time(sec)
Case1	Python	4.347	0.859	2.598
	R	4.347	0.859	1.120
Case2	Python	5.654	0.761	4.308
	R	5.654	0.761	3.265
case3 (grid search)	Python	4.099	0.874	321.376 for gridsearch
				3.809 for optimal value
	R	4.113	0.873	168.06 for gridsearch
				2.945 for optimal value

표 5 Gas turbine emission xgboost prediction case별 실행 결과

다음 두 페이지는 1. R과 Python에서 각 case 별 실제 타겟 변수의 값과 예측된 타겟 변수의 값에 대한 그래프 2. case 별 변수 중요도 그래프이다.

Case 별 observed y와 fitted y 그래프를 보면, 크게 차이가 나진 않지만 RMSE가 가장 높은 case2가 다른 두 case에 비해 y=x 선에서 좀 더 떨어져 있는 것을 알 수 있다.

Case 별 변수 중요도를 구하는 몇 가지 지표 중, 모든 트리의 분할에서 해당 feature가 사용된 횟수를 기준으로 한 F-score(R에서는 Frequency)를 기준으로 보았다. 주로 주변 온도를 뜻하는 AT 변수와 공기 필터 내 차압을 뜻하는 AFDP 변수가 중요도 상위권에 위치하였다.

Observed y vs fitted y plot in Python

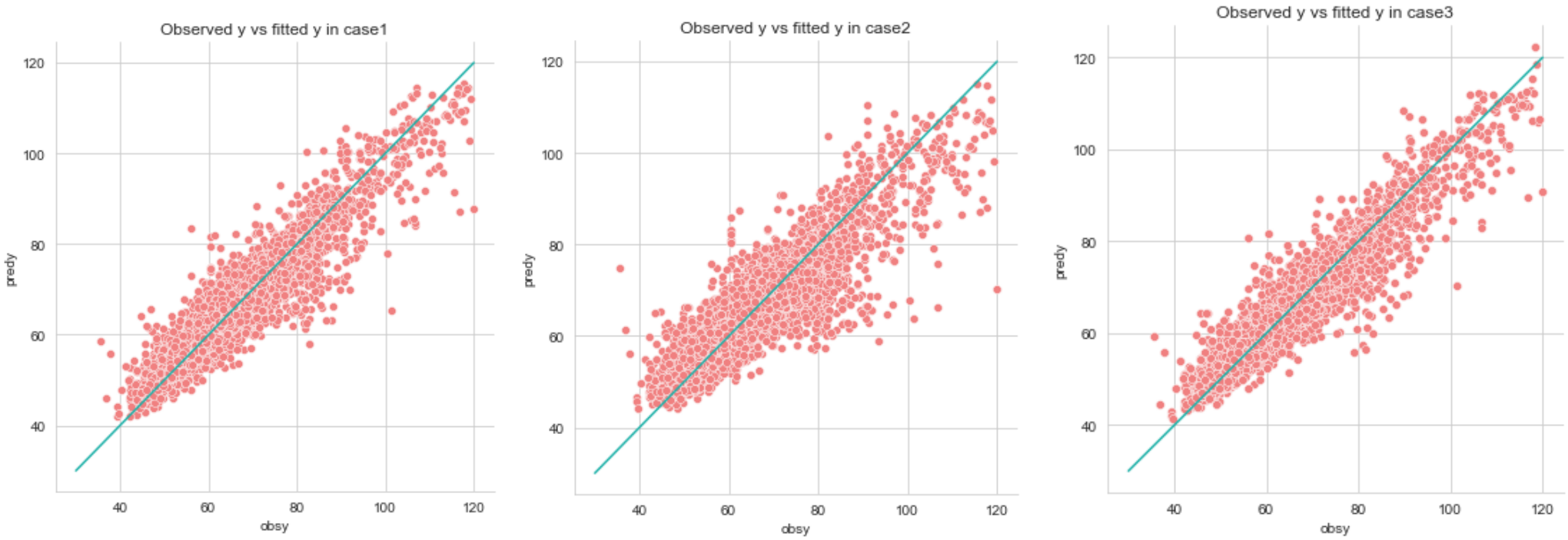


그림 6 Python에서의 observed y vs fitted y plot

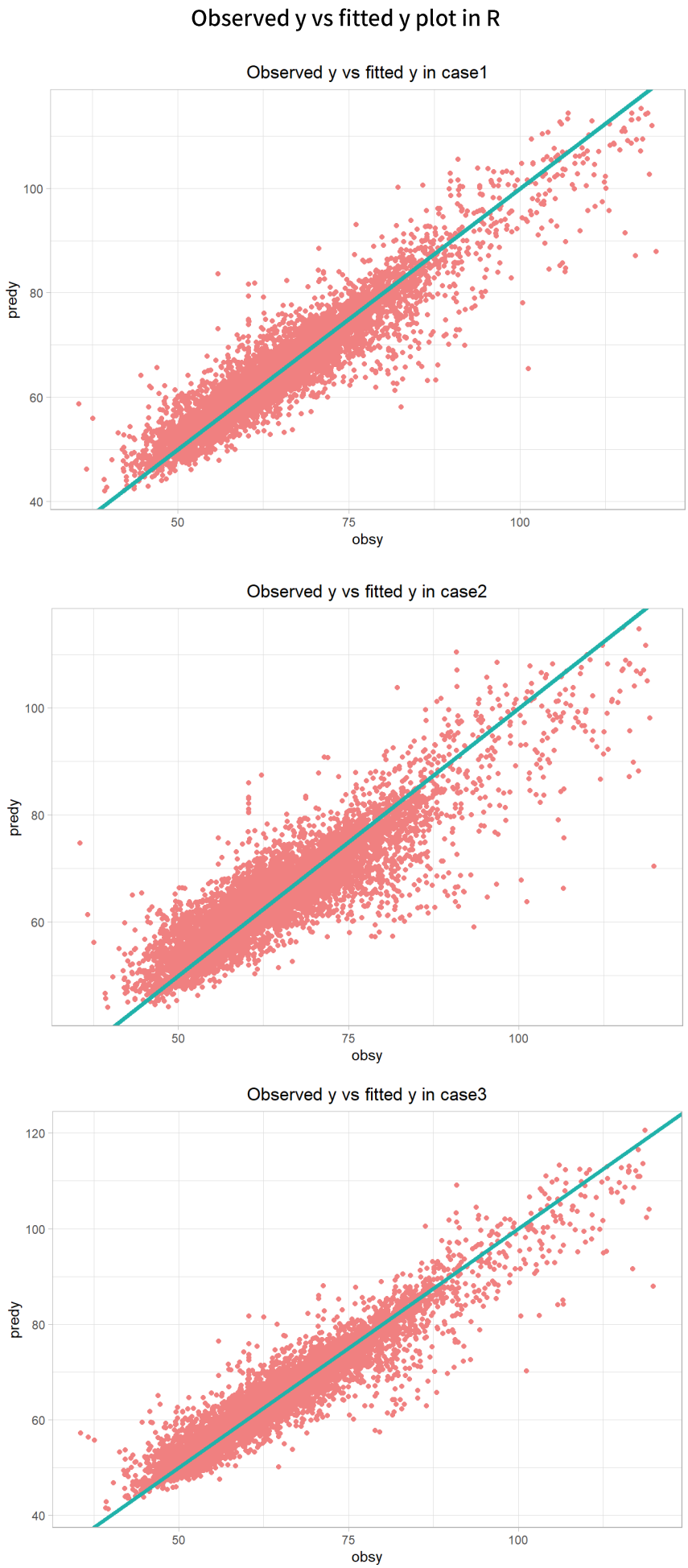


그림 7 R에서의 observed y vs fitted y plot

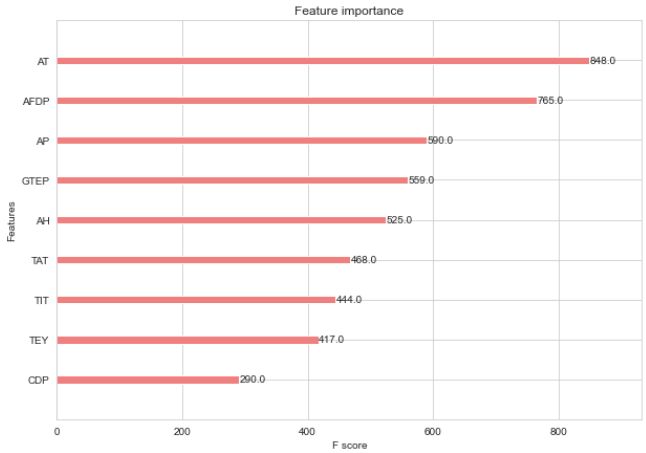
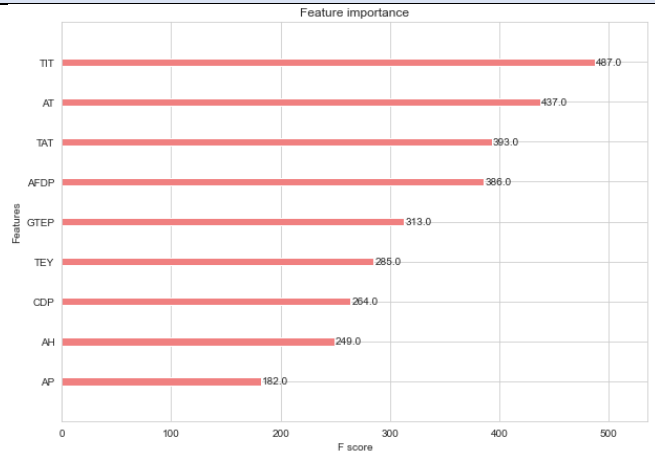
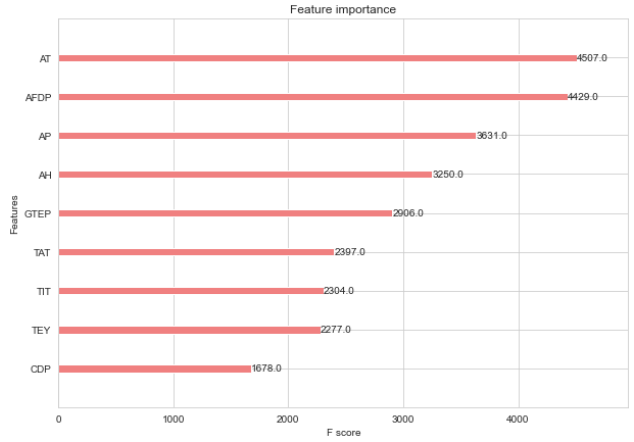
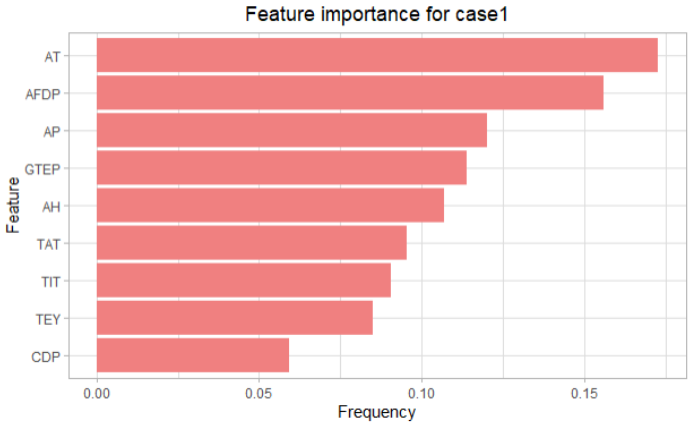
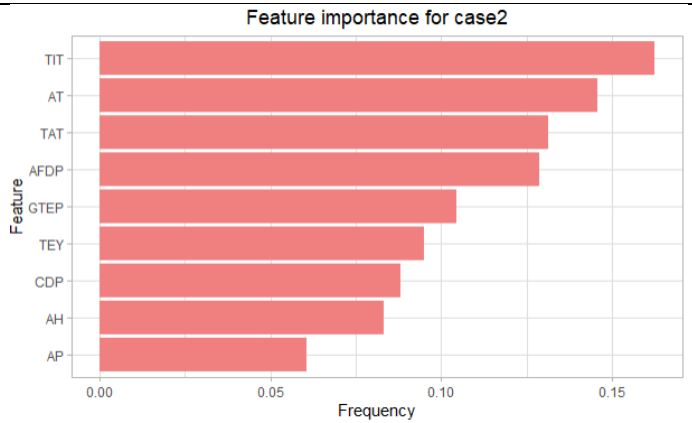
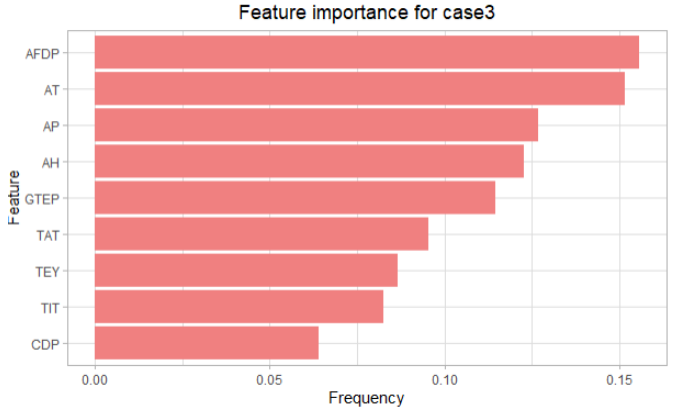
	Case 1	Case 2	Case 3
Python			
R			

표 6 Gas turbine emission xgboost prediction case별 feature importance plot

세 case의 feature importance plot에서 중요한 영향을 끼치는 것으로 확인된 AT 변수와 AFDP 변수에 대해 partial dependence plot(PDP)를 시각화하였다. PDP는 관심 변수의 각 값에 따라 타겟 변수에 어느 정도의 marginal effect를 주는지 확인할 수 있는 plot이다. AT 변수의 PDP에 해당하는 밑의 왼쪽 그림을 보면, 그래프가 우하향 하는 것을 고려했을 때 AT가 커질수록 NOX 배출량이 낮게 예측되는 것으로 볼 수 있다. 밑의 오른쪽 그래프에 해당하는 AFDP의 경우, 작은 AFDP 값에서는 그래프가 오르락 내리락 하다가 3.5 근처에서 우상향 하는 모습을 보인다. AFDP가 일정 수준 이상일 때, AFDP가 커질수록 NOX 배출량이 높게 예측된다고 볼 수 있다.

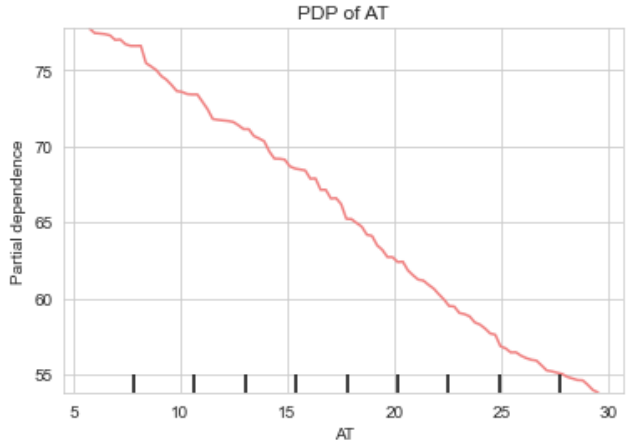
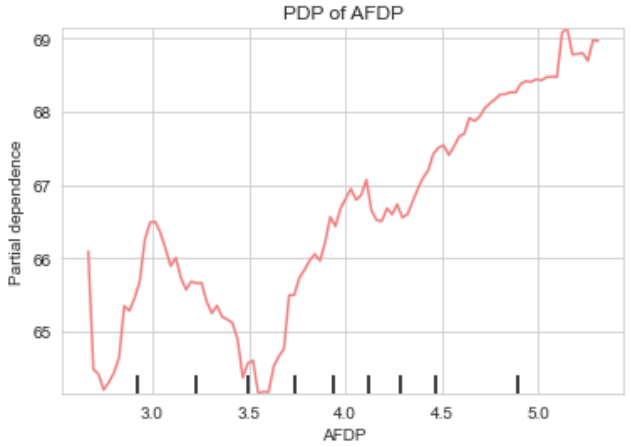
AT	AFDP
	

표 7 AT, AFDP 변수에 대한 partial dependence plot

CatBoost Implementation with R and Python

R과 Python을 통해 CatBoost를 이용한 NOX 배출량의 예측을 3가지 케이스로 나눠 진행한다.

Case 1에서는 별도의 하이퍼파라미터 튜닝을 거치지 않은 XGBoost 기본 파라미터를 그대로 사용하였고, case 2에서는 case1과 비교했을 때 각 트리의 depth는 작게 하고 learning rate는 작게 하여 모델을 적합하였다. Case 3에서는 gridsearch를 통해 몇 가지 파라미터에 대한 후보군을 두어 가장 좋은 성능을 내는 파라미터 조합을 찾고, 이를 이용하여 모델을 적합하였다.

추가적으로, CatBoost의 핵심 기능 중 하나인 ordered boosting을 사용했을 때 성능 개선의 여부를 확인해 보기 위해, ordered boosting을 사용한 경우(boosting_type="Plain")와 사용하지 않은 경우(boosting_type="Ordered)를 각각 실행하고 결과를 비교하였다.

위의 분석 케이스들을 간단히 요약하면 다음과 같다.

I : Plain boosting + case1
II : Ordered boosting + case1
III: Plain boosting + case2
IV: Ordered boosting + case2
V : Plain boosting + case3
VI: Ordered boosting + case3

[참고] 여섯가지 분석 방법에 대한 요약

각 case에서 사용한 파라미터와 case 3의 optimal value에 대한 결과는 다음 표와 같다. Catboost는 learning rate를 따로 설정하지 않을 경우 iteration에 맞춰 모델 내에서 자동적으로 learning rate를 설정한다. 따라서 case 1의 learning_rate는 catboost의 default iteration 횟수에 따라 자동적으로 결정된 값이다.

Case 3을 보면 plain boosting을 사용한 경우에는 l2_leaf_reg과 border_count가 R과 Python에서 차이가 있었고 다른 파라미터는 같은 optimal value를 얻었다. Ordered boosting의 경우, depth가 다른 값으로 설정되었고 나머지 파라미터는 모두 같은 값을 얻었다.

case	parameter	values	Optimal value	
			R	Python
case1	depth	8		
	learning_rate	iteration에 따라 자동 설정	0.087	0.069
	iterations	1000		
case2	depth	2		
	learning_rate	0.05		
	iterations	1000		
case3 (girdsearch)	depth	[4, 6, 8]	8 for plain 8 for ordered	8 for plain 6 for ordered
	learning_rate	[0.01, 0.1]	0.1	0.1
	iterations	[100, 200]	200	200
	l2_leaf_reg	[0.1, 1, 3]	0.1	1 for plain 0.1 for ordered
	border_count	[150, 254]	254	150 for plain 254 for ordered
	rsm	[0.5, 0.85]	0.85	0.85

표 8 Gas turbine emission Catboost prediction 사용 파라미터

세 case 각각에 대해 plain boosting과 ordered boosting으로 모델을 실행하고 test set을 예측한 후 계산한 RMSE와 Pseudo-R^2 및 실행시간은 밑의 표와 같다. 먼저 case별로 비교해보면, 3개 케이스 중 별도의 파라미터 튜닝 과정을 거치지 않은 case 1이 RMSE가 가장 낮고 R^2는 가장 큰 값으로 좋은 성능을 보였다.

Boosting type별로 비교해보면, 세 case 모두 Ordered boosting을 사용한 경우보다 plain을 사용했을 때의 RMSE가 모든 경우에서 더 낮았다. R과 Python 결과를 비교해보면, 같은 파라미터를 사용한 case 2는 R과 Python이 같은 성능을 보였다. 모델 학습 과정에서 자동적으로 설정된 learning rate가 달랐던 case 1의 경우 Python보다 R이 좀 더 좋은 성능을 보였다. Case 3의 ordered boosting의 경우 Python과 R은 트리 깊이(depth)의 차이가 있었는데, depth=6이었던 Python보다 depth=8인 R의 RMSE가 더 낮았다.

모든 case를 고려했을 때, R에서 default 파라미터를 이용한 case 1 중 plain boosting을 이용한 모델이 가장 좋은 성능을 보였다고 판단하였다. 실행시간의 경우 case 1과 2는 R, case 3은 Python이 더 빨랐다.

case	boosting type	Tool	RMSE	Pseudo-R^2	time(sec)
Case1	Plain	Python	4.231	0.866	5.184
		R	4.149	0.871	4.357
	Ordered	Python	4.373	0.857	16.356
		R	4.311	0.861	15.913
Case2	Plain	Python	5.827	0.746	2.646
		R	5.827	0.746	2.520
	Ordered	Python	5.861	0.743	6.386
		R	5.861	0.743	5.592
case3 (grid search)	Plain	Python	4.572	0.844	148.10 for gridsearch
					2.192 for optimal value
		R	4.535	0.846	259.98 for gridsearch
					3.351 for optimal value
	Ordered	Python	4.931	0.818	291.59 for gridsearch
					4.614 for optimal value
		R	4.658	0.838	759.65 for gridsearch
					10.10 for optimal value

표 9 Gas turbine emission catboost prediction case별 실행 결과

다음 여섯 페이지는 R과 Python에서 각 case 별 실제 타겟 변수의 값과 예측된 타겟 변수의 값에 대한 그래프와 변수 중요도 그래프이다. 먼저 observed y vs fitted y 그래프를 보면 케이스별로 큰 차이가 나는 것은 아니지만 가장 RMSE가 낮은 case 1의 점들이 x=y선에서 제일 덜 퍼져 있는 것으로 보인다.

Feature importance plot을 보면 모든 케이스에서 AT 변수가 가장 중요도가 높은 것을 알 수 있고 AT 이외에 TIT, TAT 변수 등이 주로 상위권에 속한다.

Observed y vs Fitted y - Case 1

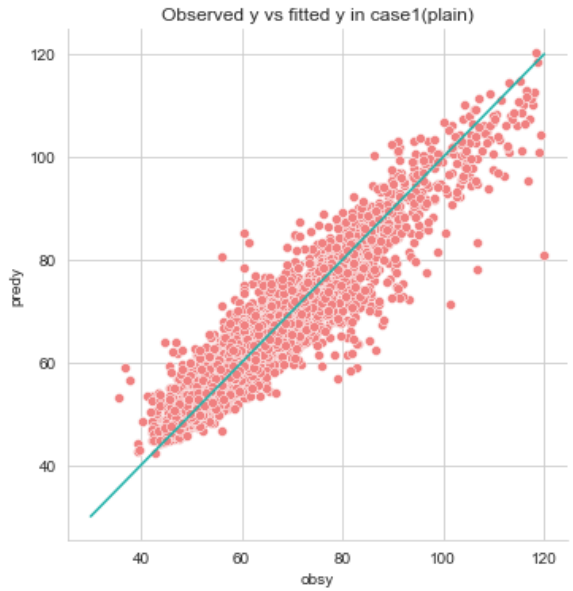
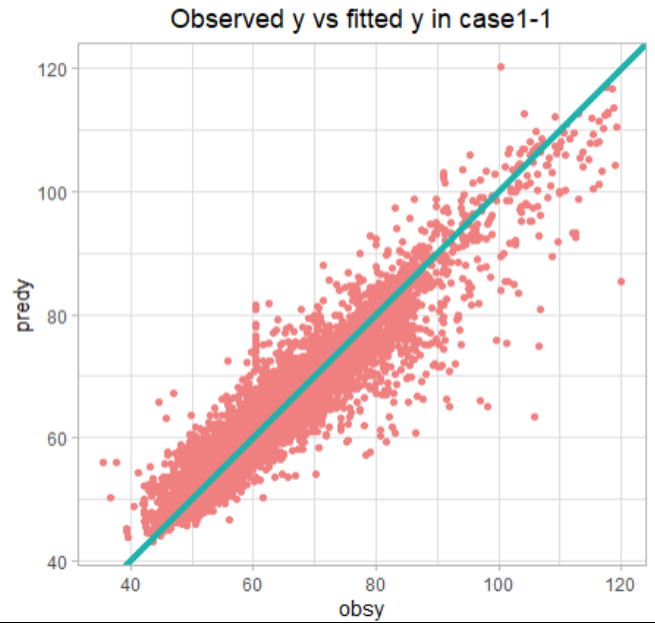
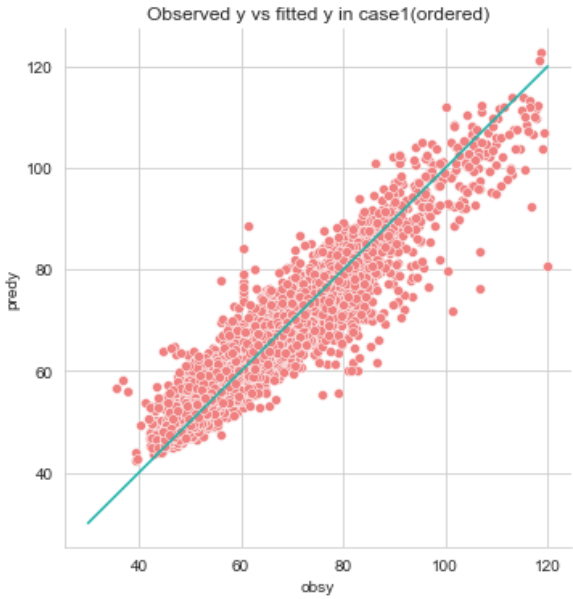
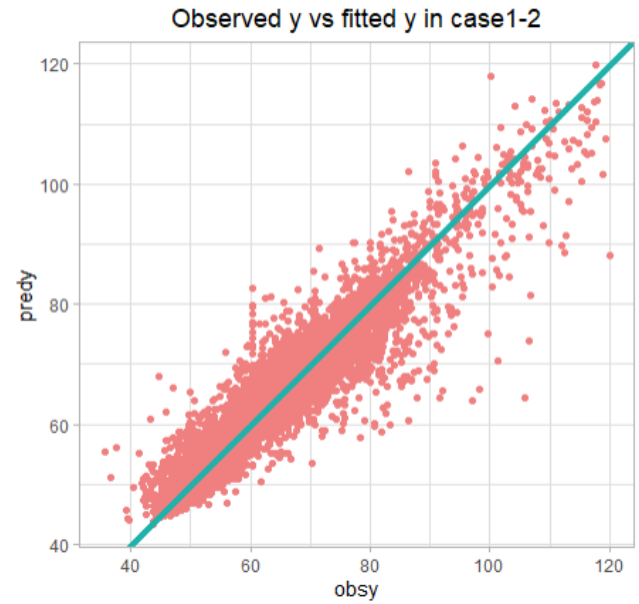
	Python	R
Plain		
Ordered		

표 10 Gas turbine emission prediction observed y vs fitted y plot – case 1

Observed y vs Fitted y - Case 2

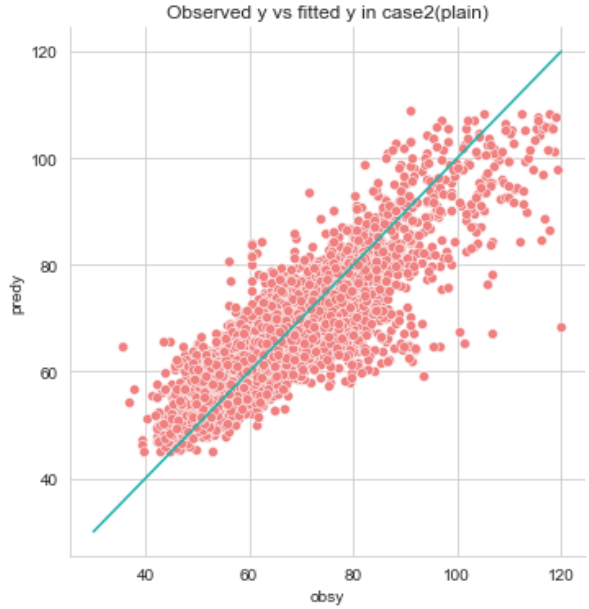
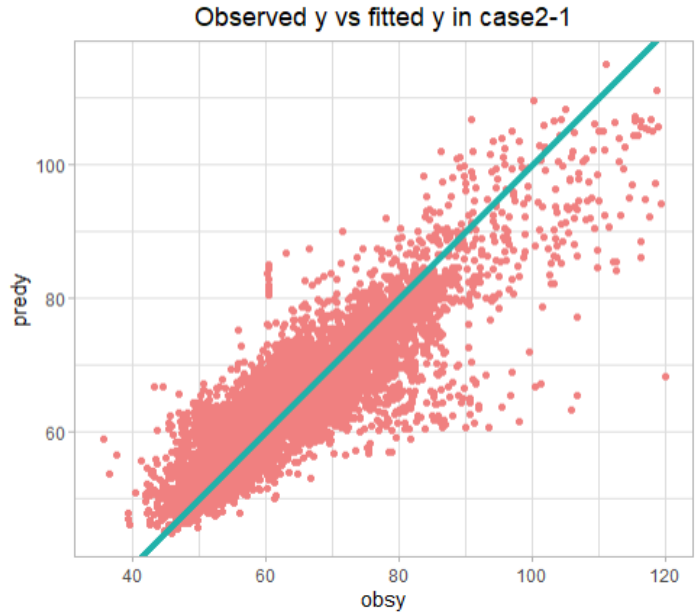
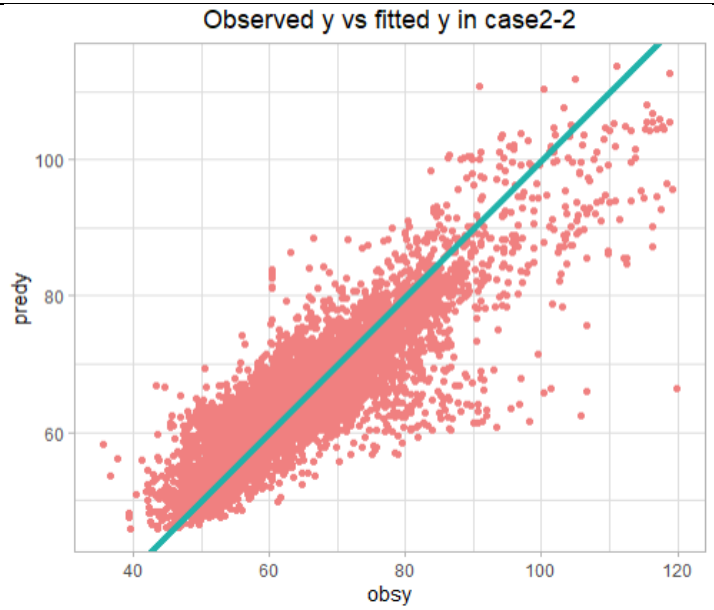
	Python	R
Plain		
Ordered		

표 11 Gas turbine emission prediction observed y vs fitted y plot – case 2

Observed y vs Fitted y - Case 3

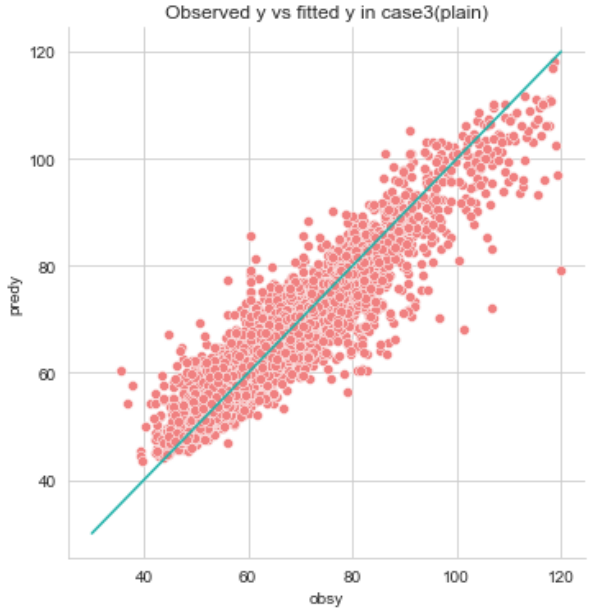
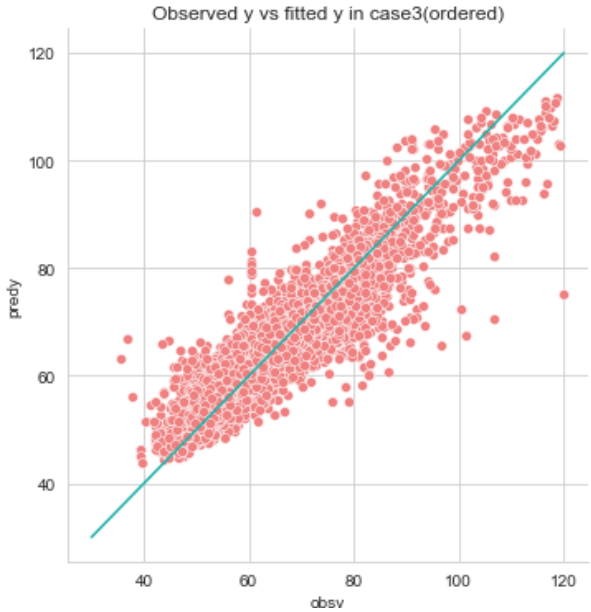
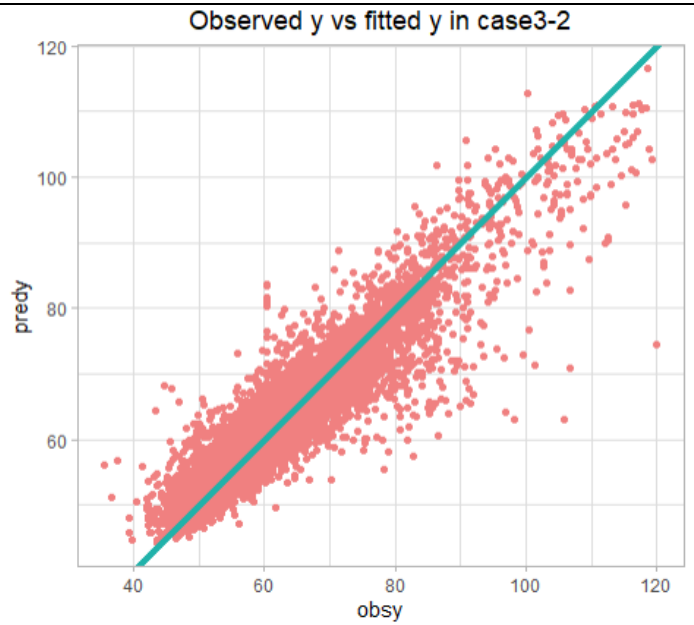
	Python	R
Plain		
Ordered		

표 12 Gas turbine emission prediction observed y vs fitted y plot – case 3

Feature importance plot - Case 1

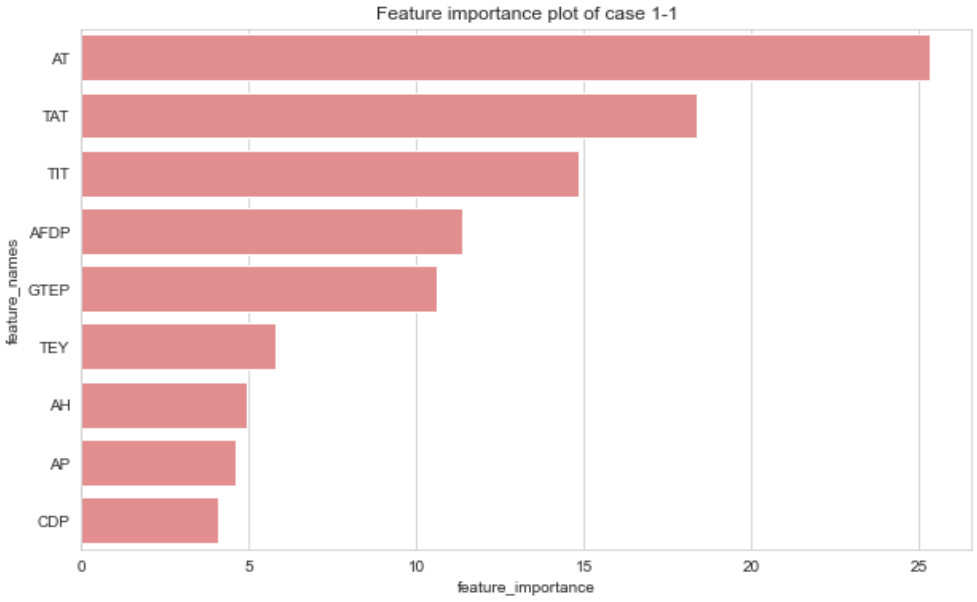
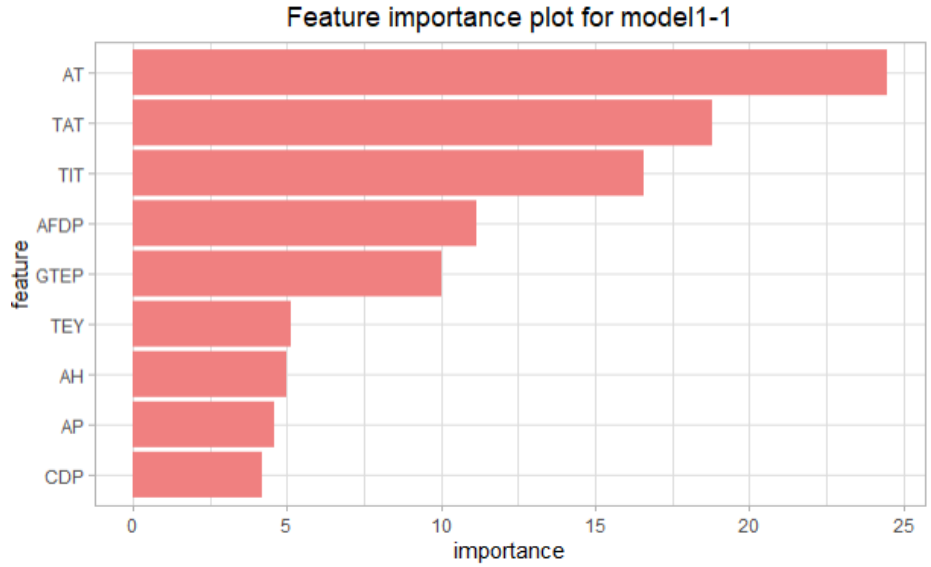
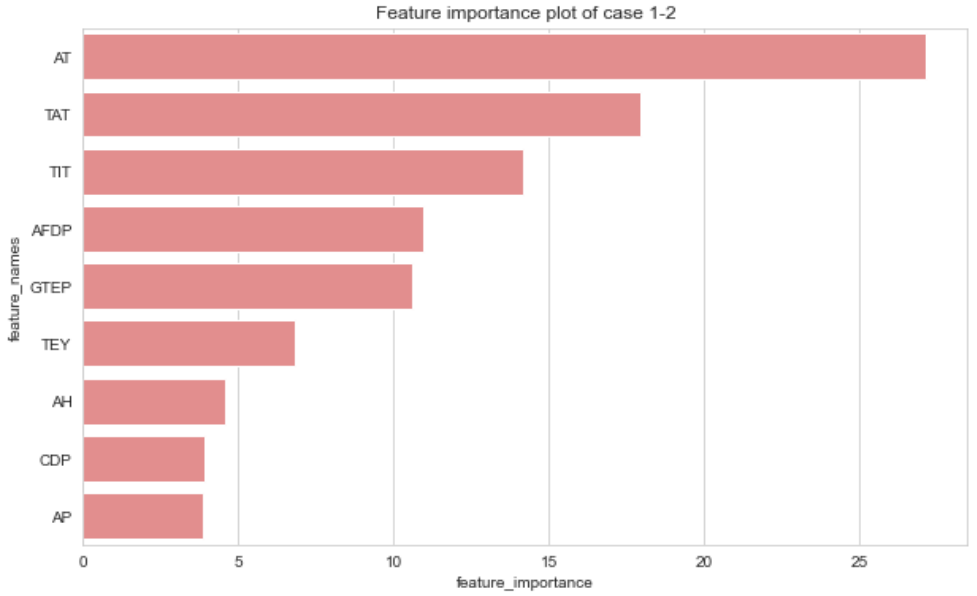
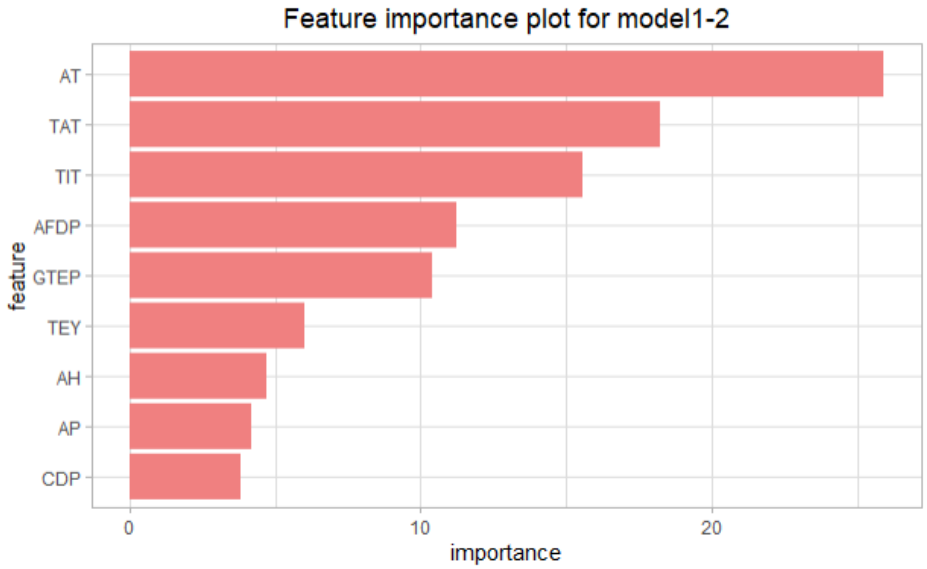
	Python	R
Plain		
Ordered		

表 13 Gas turbine emission prediction feature importance plot – case 1

Feature importance plot - Case 2

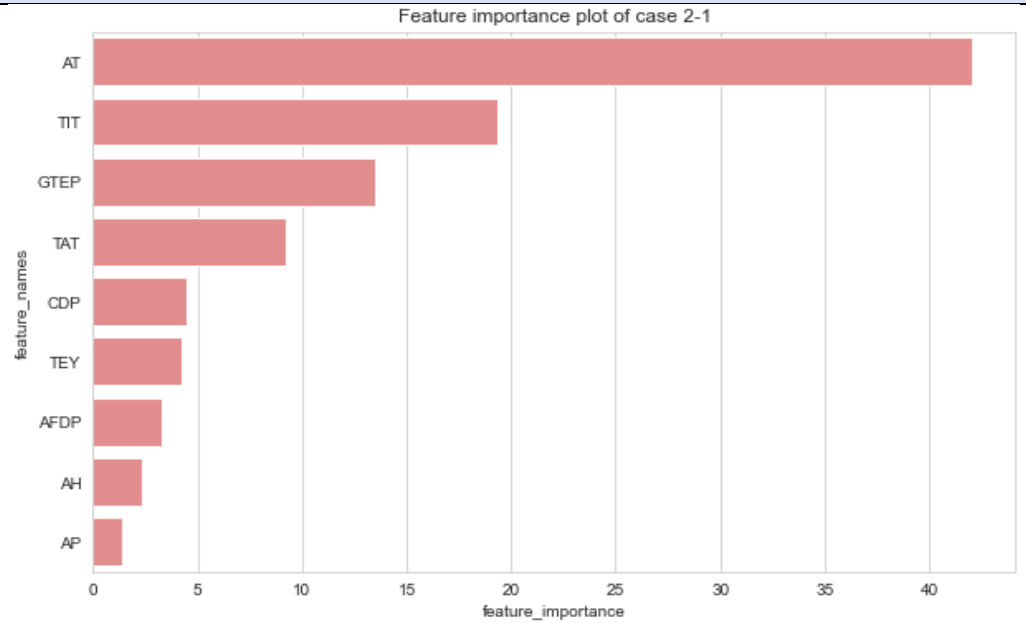
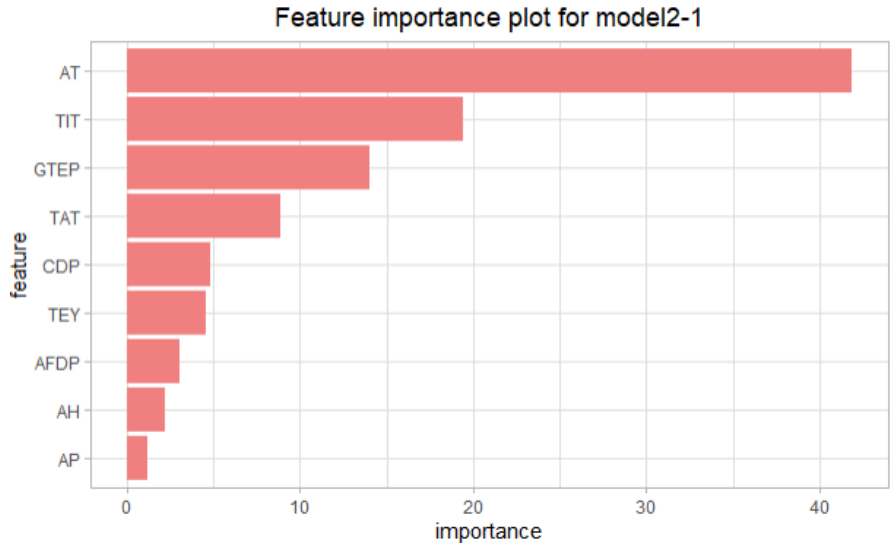
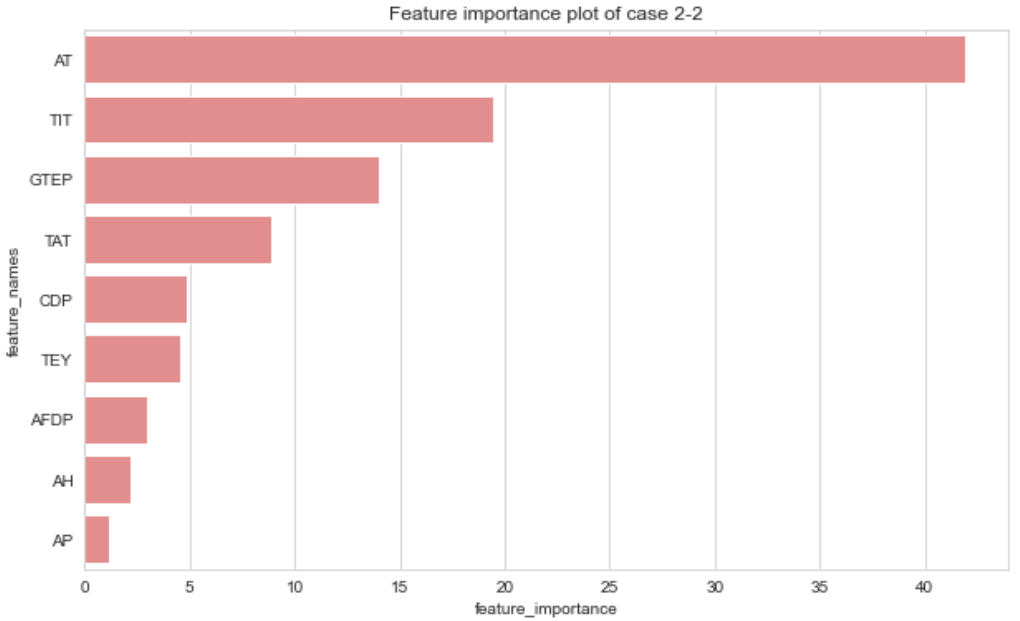
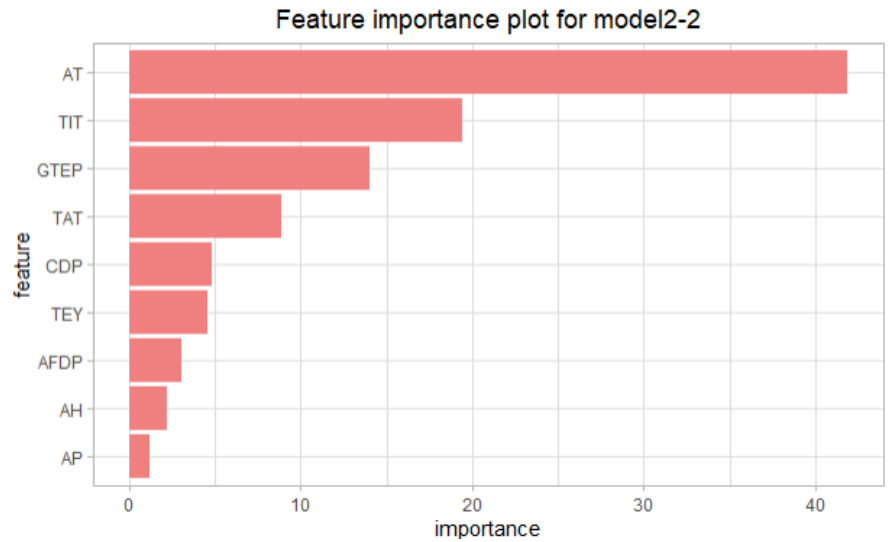
	Python	R
Plain		
Ordered		

表 14 Gas turbine emission prediction feature importance plot – case 2

Feature importance plot - Case 3

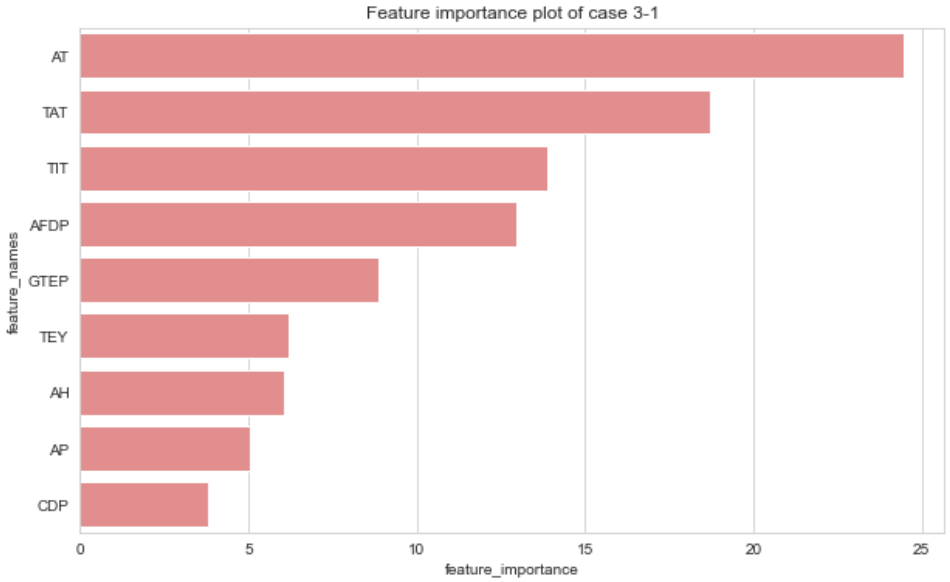
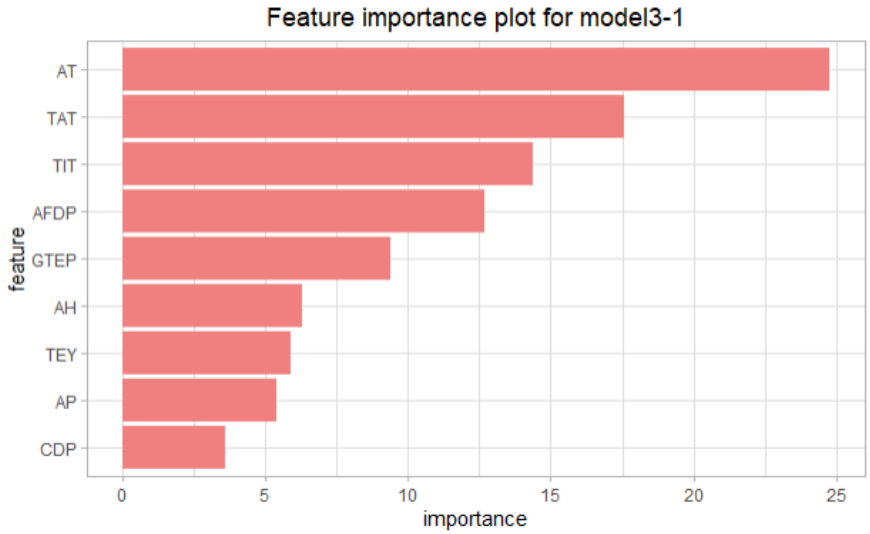
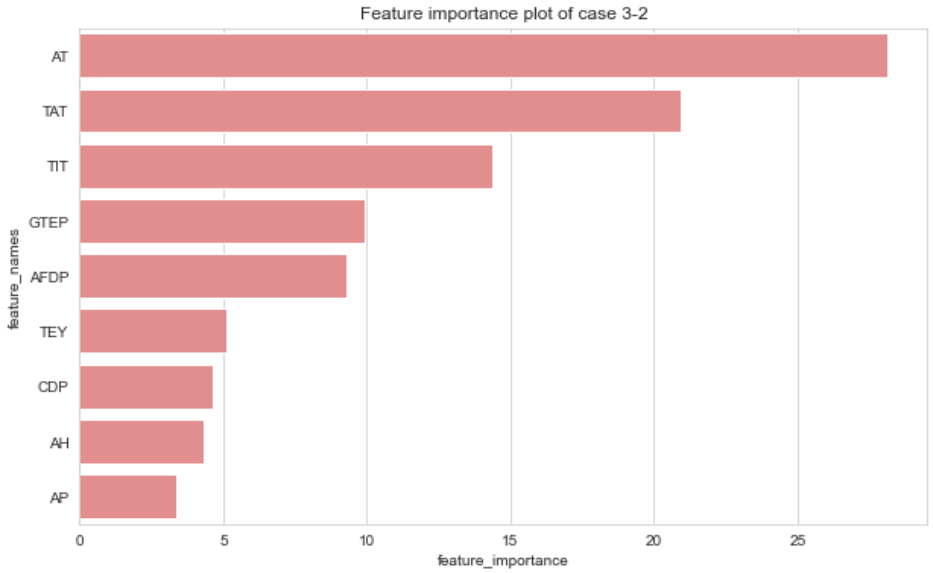
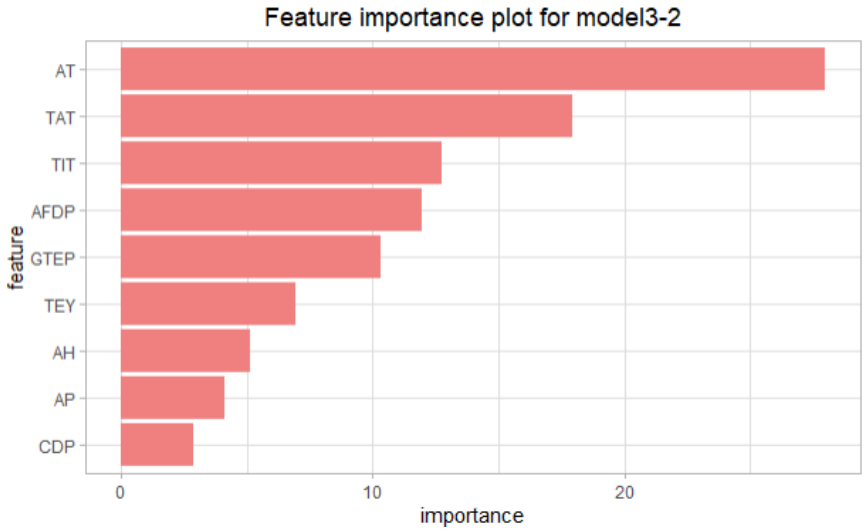
	Python	R
Plain		
Ordered		

表 15 Gas turbine emission prediction feature importance plot – case 3

세 case의 feature importance plot에서 중요한 영향을 끼치는 것으로 확인된 AT, TAT, TIT 변수에 대해 partial dependence plot(PDP)를 시각화하였다. 표 17의 위의 두 그래프에 해당하는 AT, TAT 변수의 경우, 그래프가 우하향 하는 것을 고려했을 때 AT와 TAT가 커질수록 NOX 배출량이 낮게 예측되는 것으로 볼 수 있다. 반대로 밑의 왼쪽 그래프에 해당하는 TIT의 경우 그래프가 우상향하는 모습을 보인다. TIT가 커질수록 NOX 배출량이 높게 예측된다고 볼 수 있다.

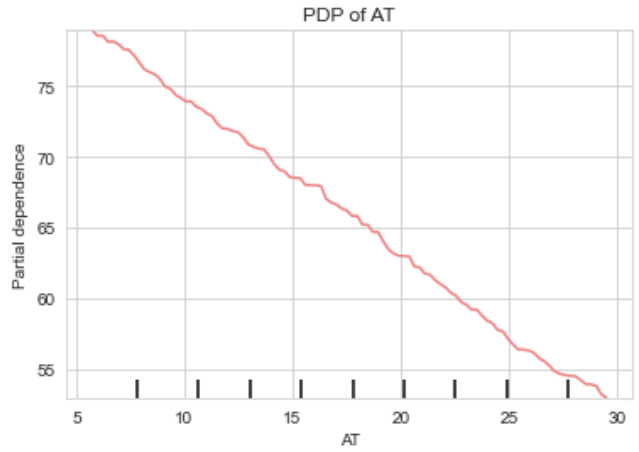
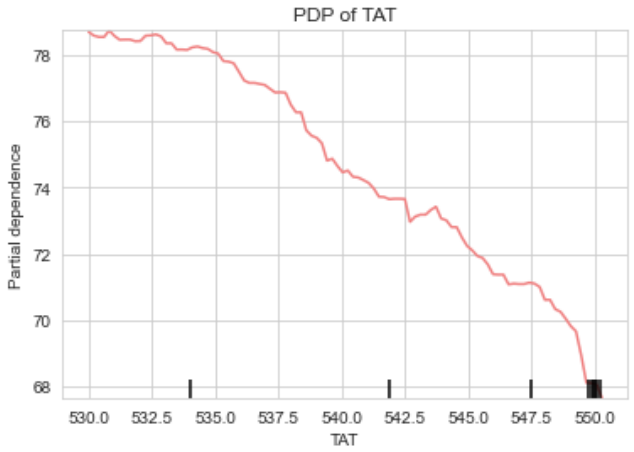
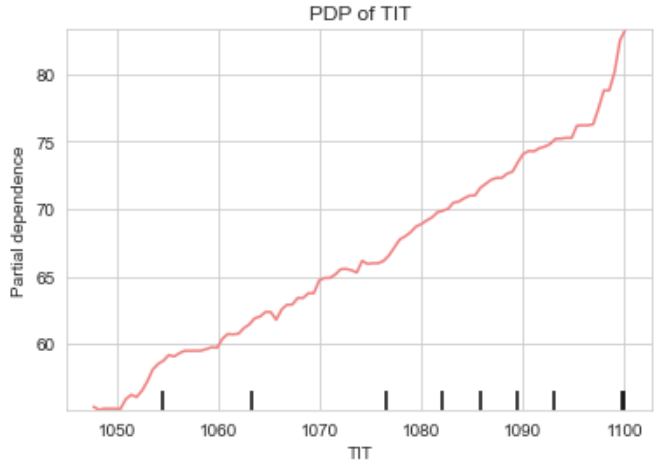
AT	TAT
	
TIT	
	

표 16 AT, TAT, TIT 변수의 partial dependence plot

XGBoost vs CatBoost

XGBoost와 CatBoost를 통한 예측 결과를 비교해본다. 밑의 표는 XGBoost와 CatBoost를 통한 예측에 대한 RMSE 값이다. 기본 모델에 파라미터 튜닝을 하지 않은 case 1의 경우, plain boosting을 사용한 catboost의 성능이 가장 좋았다. 반면 파라미터 튜닝이 들어간 case 2와 3은 xgboost의 성능이 좋은 것을 확인할 수 있다. 모든 케이스를 고려하였을 때 Python에서 grid search를 통해 찾은 파라미터를 이용하여 xgboost를 적합했을 때의 결과가 가장 좋았다.

case	Tool	XGBoost	CatBoost-Plain	Catboost-Ordered
Case1	Python	4.347	4.231	4.373
	R	4.347	4.149	4.311
Case2	Python	5.654	5.827	5.861
	R	5.654	5.827	5.861
case3 (grid search)	Python	4.099	4.572	4.931
	R	4.113	4.535	4.658

표 17 Gas turbine emission prediction RMSE 모델별 비교

Appendix

Python code

```
import pandas as pd

import numpy as np

import matplotlib.pyplot as plt

import seaborn as sns

import random

import time

import warnings

warnings.filterwarnings('ignore')

sns.set_style("whitegrid")

from sklearn.model_selection import train_test_split, GridSearchCV


import xgboost

from xgboost import plot_importance

from catboost import CatBoostRegressor, Pool, metrics, cv


from sklearn.metrics import mean_squared_error

from sklearn.metrics import precision_score

from sklearn.metrics import recall_score

from sklearn.metrics import f1_score

from sklearn.metrics import classification_report

from sklearn.metrics import plot_confusion_matrix


d1 = pd.read_csv('./pp_gas_emission/gt_2011.csv')

d2 = pd.read_csv('./pp_gas_emission/gt_2012.csv')
```

```
d3 = pd.read_csv('./pp_gas_emission/gt_2013.csv')

d4 = pd.read_csv('./pp_gas_emission/gt_2014.csv')

d5 = pd.read_csv('./pp_gas_emission/gt_2015.csv')

d1.head()


# 데이터 병합

data = pd.concat([d1, d2, d3, d4, d5], axis=0)

data.info()


data['NOX'].describe().T


plt.figure(figsize=(16, 5))

sns.histplot(data['NOX'], color="lightcoral", bins=50)

plt.title("Distibution of NOX(Target variable)");


cols = data.columns[:10]

fig = plt.figure(figsize=(22, 9))

for i in range(10):

    plt.subplot(2, 5, i+1)

    sns.histplot(data[cols[i]], color="lightcoral")

    plt.title("Distribution of "+cols[i]);


X = data.drop('NOX', axis=1)

y = data['NOX']

train_x, test_x, train_y, test_y = train_test_split(X, y, test_size=0.2, random_state=777)

train_x.head()
```

```
# train = pd.concat([train_x, train_y], axis=1)

# test = pd.concat([test_x, test_y], axis=1)

# train.to_csv('./pp_gas_emission/train.csv', index=False)

# test.to_csv('./pp_gas_emission/test.csv', index=False)



# ### xgboost

# #### Modeling 1: xgboost 기본 파라미터 이용


start = time.time()

xgb = xgboost.XGBRegressor()

xgb.fit(train_x, train_y)

print("실행시간:{}".format(time.time()-start))


pred_y = xgb.predict(test_x)

rmse1 = np.sqrt(mean_squared_error(test_y, pred_y))

rmse1


# observed y vs fitted y

y_df = pd.DataFrame({'obsy':test_y, 'predy':pred_y})

sns.relplot(x='obsy', y='predy', data=y_df, color="lightcoral")

plt.plot([30, 120], [30, 120], color="lightseagreen")

plt.title("Observed y vs fitted y in case1");


# pseudo R^2
```

```
1 - sum((test_y - pred_y)**2) / sum((test_y - np.mean(test_y))**2)


fig, ax = plt.subplots(1, 1, figsize=(10, 7))

plot_importance(xgb, ax=ax, color="lightcoral");



# #### Modeling 2: 파라미터 조정 - depth와 learning rate는 작게, 트리 수는 많게

start = time.time()

xgb2 = xgboost.XGBRegressor(max_depth=2, learning_rate=0.05, n_estimators=1000)

xgb2.fit(train_x, train_y)

print("실행시간:{}".format(time.time()-start))


pred_y2 = xgb2.predict(test_x)

rmse2 = np.sqrt(mean_squared_error(test_y, pred_y2))

rmse2


# observed y vs fitted y

y_df = pd.DataFrame({'obsy':test_y, 'predy':pred_y2})

sns.relplot(x='obsy', y='predy', data=y_df, color="lightcoral")

plt.plot([30, 120], [30, 120], color="lightseagreen")

plt.title("Observed y vs fitted y in case2");


# pseudo R^2

1 - sum((test_y - pred_y2)**2) / sum((test_y - np.mean(test_y))**2)
```

```
fig, ax = plt.subplots(1, 1, figsize=(10, 7))
plot_importance(xgb2, ax=ax, color="lightcoral");



# #### Modeling 3: Gridsearch
start = time.time()
xgb3 = xgboost.XGBRegressor()
xgb_grid = {
    'colsample_bytree':[0.6, 0.8],
    'gamma': [0.5, 0.75, 1],
    'n_estimators':[100, 200],
    'learning_rate':[0.01, 0.1],
    'max_depth':[4, 6, 8],
    'random_state':[777]
}
xgbCV = GridSearchCV(xgb3, xgb_grid, verbose=2, cv=3)
xgbCV.fit(train_x, train_y)
print("실행시간:{}".format(time.time()-start))
print(xgbCV.best_params_)


# best parameter  사용
start = time.time()
xgb3 = xgboost.XGBRegressor(colsample_bytree=0.8, gamma=0.75, learning_rate=0.1,
                            max_depth=8,
                             n_estimators=200,
                             random_state=777)
```

```
xgb3.fit(train_x, train_y)

print("실행시간:{}".format(time.time()-start))


pred_y3 = xgb3.predict(test_x)

rmse3 = np.sqrt(mean_squared_error(test_y, pred_y3))

rmse3


# observed y vs fitted y

y_df = pd.DataFrame({'obsy':test_y, 'predy':pred_y3})

sns.relplot(x='obsy', y='predy', data=y_df, color="lightcoral")

plt.plot([30, 120], [30, 120], color="lightseagreen")

plt.title("Observed y vs fitted y in case3");


# pseudo R^2

1 - sum((test_y - pred_y3)**2) / sum((test_y - np.mean(test_y))**2)


fig, ax = plt.subplots(1, 1, figsize=(10, 7))

plot_importance(xgb3, ax=ax, color="lightcoral");



# ### CatBoost

# #### Modeling 1: catboost default  파라미터  사용


# * 1-1.Plain

start = time.time()

model1_1 = CatBoostRegressor(
```

```
                                    loss_function="RMSE")
model1_1.fit(train_x, train_y)
print("실행시간:{}".format(time.time()-start))

print(model1_1.get_all_params())

pred_y11 = model1_1.predict(test_x)
rmse1_1 = np.sqrt(mean_squared_error(test_y, pred_y11))
rmse1_1

# observed y vs fitted y
y_df = pd.DataFrame({'obsy':test_y, 'predy':pred_y11})
sns.relplot(x='obsy', y='predy', data=y_df, color="lightcoral")
plt.plot([30, 120], [30, 120], color="lightseagreen")
plt.title("Observed y vs fitted y in case1(plain)");

# pseudo R^2
1 - sum((test_y - pred_y11)**2) / sum((test_y - np.mean(test_y))**2)

temp = pd.DataFrame({'feature_importance': model1_1.get_feature_importance(),
                     'feature_names': train_x.columns}).sort_values(by=['feature_importance'],
                                                                    ascending=False)

fig = plt.figure(figsize=(10, 6))
sns.barplot(x='feature_importance', y='feature_names', data=temp, color="lightcoral")
plt.title("Feature importance plot of case 1-1");
```

```
# * 1-2. Ordered


start = time.time()

model1_2 = CatBoostRegressor(

                                boosting_type="Ordered",

                                loss_function="RMSE")

model1_2.fit(train_x, train_y)

print("실행시간:{}".format(time.time()-start))


pred_y12 = model1_2.predict(test_x)

rmse1_2 = np.sqrt(mean_squared_error(test_y, pred_y12))

rmse1_2


# observed y vs fitted y

y_df = pd.DataFrame({'obsy':test_y, 'predy':pred_y12})

sns.relplot(x='obsy', y='predy', data=y_df, color="lightcoral")

plt.plot([30, 120], [30, 120], color="lightseagreen")

plt.title("Observed y vs fitted y in case1(ordered)");


# pseudo R^2

1 - sum((test_y - pred_y12)**2) / sum((test_y - np.mean(test_y))**2)


temp = pd.DataFrame({'feature_importance': model1_2.get_feature_importance(),

                     'feature_names': train_x.columns}).sort_values(by=['feature_importance'],

                                                                    ascending=False)
```

```
fig = plt.figure(figsize=(10, 6))

sns.barplot(x='feature_importance', y='feature_names', data=temp, color="lightcoral")

plt.title("Feature importance plot of case 1-2");



# #### Modeling 2:  파라미터  조정


# * 2-1. Plain


start = time.time()

model2_1 = CatBoostRegressor(random_seed=777,

                                        depth=2,

                                        learning_rate=0.05,

                                        iterations=1000,

                                        loss_function="RMSE")

model2_1.fit(train_x, train_y)

print("실행시간:{}".format(time.time()-start))


pred_y21 = model2_1.predict(test_x)

rmse2_1 = np.sqrt(mean_squared_error(test_y, pred_y21))

rmse2_1


# observed y vs fitted y

y_df = pd.DataFrame({'obsy':test_y, 'predy':pred_y21})

sns.relplot(x='obsy', y='predy', data=y_df, color="lightcoral")
```

```
plt.plot([30, 120], [30, 120], color="lightseagreen")
plt.title("Observed y vs fitted y in case2(plain)");

# pseudo R^2
1 - sum((test_y - pred_y21)**2) / sum((test_y - np.mean(test_y))**2)

temp = pd.DataFrame({'feature_importance': model2_1.get_feature_importance(),
                     'feature_names': train_x.columns}).sort_values(by=['feature_importance'],
                                                                    ascending=False)

fig = plt.figure(figsize=(10, 6))
sns.barplot(x='feature_importance', y='feature_names', data=temp, color="lightcoral")
plt.title("Feature importance plot of case 2-1");

# * 2-2.Ordered
start = time.time()
model2_2 = CatBoostRegressor(random_seed=777,
                             depth=2,
                             learning_rate=0.05,
                             iterations=1000,
                             boosting_type="Ordered",
                             loss_function="RMSE")
model2_2.fit(train_x, train_y)
print("실행시간:{}".format(time.time()-start))

pred_y22 = model2_2.predict(test_x)
```

```
rmse2_2 = np.sqrt(mean_squared_error(test_y, pred_y22))

rmse2_2


# observed y vs fitted y

y_df = pd.DataFrame({'obsy':test_y, 'predy':pred_y22})

sns.relplot(x='obsy', y='predy', data=y_df, color="lightcoral")

plt.plot([30, 120], [30, 120], color="lightseagreen")

plt.title("Observed y vs fitted y in case2(ordered)");


# pseudo R^2

1 - sum((test_y - pred_y22)**2) / sum((test_y - np.mean(test_y))**2)


temp = pd.DataFrame({'feature_importance': model2_2.get_feature_importance(),

                     'feature_names': train_x.columns}).sort_values(by=['feature_importance'],

                                                                    ascending=False)


fig = plt.figure(figsize=(10, 6))

sns.barplot(x='feature_importance', y='feature_names', data=temp, color="lightcoral")

plt.title("Feature importance plot of case 2-2");



# #### Modeling 3: Gridsearch

# * 3-1. Plain

model = CatBoostRegressor(random_seed=777)

grid = {'learning_rate': [0.01,0.1],

        'depth': [4, 6, 8],
```

```
            'l2_leaf_reg': [0.1, 1, 3],
            'iterations':[100, 200],
            'border_count':[150,254],
            'rsm':[0.5,0.85]
            }

#cv default=3
grid_search_result = model.grid_search(grid,
                                       X=train_x,
                                       y=train_y,
                                       plot=True)

grid_search_result

start = time.time()
model3_1 = CatBoostRegressor(random_seed=777,
                             depth=8,
                             learning_rate=0.1,
                             l2_leaf_reg = 0.1,
                             iterations=200,
                             rsm=0.85,
                             border_count=254,
                             loss_function="RMSE")
model3_1.fit(train_x, train_y)
print("실행시간:{}".format(time.time()-start))
```

```
pred_y31 = model3_1.predict(test_x)

rmse3_1 = np.sqrt(mean_squared_error(test_y, pred_y31))

rmse3_1


# observed y vs fitted y

y_df = pd.DataFrame({'obsy':test_y, 'predy':pred_y31})

sns.relplot(x='obsy', y='predy', data=y_df, color="lightcoral")

plt.plot([30, 120], [30, 120], color="lightseagreen")

plt.title("Observed y vs fitted y in case3(plain)");


# pseudo R^2

1 - sum((test_y - pred_y31)**2) / sum((test_y - np.mean(test_y))**2)


temp = pd.DataFrame({'feature_importance': model3_1.get_feature_importance(),

                     'feature_names': train_x.columns}).sort_values(by=['feature_importance'],

                                                                    ascending=False)


fig = plt.figure(figsize=(10, 6))

sns.barplot(x='feature_importance', y='feature_names', data=temp, color="lightcoral")

plt.title("Feature importance plot of case 3-1");


# * 3-2.Ordered

model = CatBoostRegressor(random_seed=777, boosting_type="Ordered")

grid = {'learning_rate': [0.01,0.1],

        'depth': [4, 6, 6],

        'l2_leaf_reg': [0.1, 1, 3],
```

```
            'iterations':[100, 200],
            'border_count':[150,254],
            'rsm':[0.5,0.85]
            }

#cv default=3
grid_search_result2 = model.grid_search(grid,
                                        X=train_x,
                                        y=train_y,
                                        plot=True)

grid_search_result2

start = time.time()
model3_2 = CatBoostRegressor(random_seed=777,
                             depth=6,
                             learning_rate=0.1,
                             l2_leaf_reg = 0.1,
                               iterations=200,
                             rsm=0.85,
                             border_count=150,
                             boosting_type="Ordered",
                             loss_function="RMSE")
model3_2.fit(train_x, train_y)
print("실행시간:{}".format(time.time()-start))
```

```
pred_y32 = model3_2.predict(test_x)

rmse3_2 = np.sqrt(mean_squared_error(test_y, pred_y32))

rmse3_2


# observed y vs fitted y

y_df = pd.DataFrame({'obsy':test_y, 'predy':pred_y32})

sns.relplot(x='obsy', y='predy', data=y_df, color="lightcoral")

plt.plot([30, 120], [30, 120], color="lightseagreen")

plt.title("Observed y vs fitted y in case3(ordered)");


# pseudo R^2

1 - sum((test_y - pred_y32)**2) / sum((test_y - np.mean(test_y))**2)


temp = pd.DataFrame({'feature_importance': model3_2.get_feature_importance(),

                     'feature_names': train_x.columns}).sort_values(by=['feature_importance'],

                                                                    ascending=False)


fig = plt.figure(figsize=(10, 6))

sns.barplot(x='feature_importance', y='feature_names', data=temp, color="lightcoral")

plt.title("Feature importance plot of case 3-2");
```

R code

```
#### load packages & data

setwd("C:/Users/YJ-HWANG/Desktop/스터디/위클리미팅")

library(tidyverse)

library(caret)

library(xgboost)

library(catboost)

train <- read.csv('./pp_gas_emission/train.csv')

test <- read.csv('./pp_gas_emission/test.csv')

head(train)

#### modeling을 위한 데이터 생성

train_x <- train[, 1:10]

train_y <- train$NOX

test_x <- test[, 1:10]

test_y <- test$NOX


## XGBoost

dtrain <- xgb.DMatrix(data = as.matrix(train_x), label=train_y)

dtest <- xgb.DMatrix(data = as.matrix(test_x))


#### Modeling 1

param <- list(objective     = "reg:squarederror",

              eval_metric = "rmse")


start_time <- Sys.time()

xgb <- xgb.train(params   = param,

                 data       = dtrain,
```

```
                          nrounds=100)
end_time <- Sys.time()
cat("실행시간: ", end_time-start_time)
pred_y <- predict(xgb, dtest)
rmse1 <- sqrt(mean((test_y - pred_y)^2))
rmse1
# observed y vs fitted y
y_df <- data.frame(obsy = test_y, predy=pred_y)
y_df %>% ggplot(aes(obsy, predy)) +
   geom_point(colour="lightcoral")+
   geom_abline(intercept = 0, slope=1, color="lightseagreen", size=1.5)+
   theme_light()+
   labs(title="Observed y vs fitted y in case1")+
   theme(plot.title = element_text(hjust=0.5))
# pseudo R^2
1 - sum((test_y - pred_y)**2) / sum((test_y - mean(test_y))^2)
# feature importance plot
data.frame(xgb.importance(colnames(dtrain), model = xgb)) %>%
   ggplot(aes(reorder(Feature, Frequency), Frequency))+
   geom_bar(stat="identity", fill="lightcoral")+
   coord_flip()+theme_light()+
   labs(x="Feature", title="Feature importance for case1")+
   theme(plot.title = element_text(hjust=0.5))


#### Modeling 2:  파라미터  조정
param <- list(objective      = "reg:squarederror",
                        eval_metric = "rmse",
```

```
                    max_depth      = 2,
                    eta                = 0.05)

start_time <- Sys.time()
xgb2 <- xgb.train(params    = param,
                  data       = dtrain,
                  nrounds = 1000)
end_time <- Sys.time()
cat("실행시간: ", end_time-start_time)
pred_y2 <- predict(xgb2, dtest)
rmse2 <- sqrt(mean((test_y - pred_y2)^2))
rmse2
# observed y vs fitted y
y_df <- data.frame(obsy = test_y, predy=pred_y2)
y_df %>% ggplot(aes(obsy, predy)) +
  geom_point(colour="lightcoral")+
  geom_abline(intercept = 0, slope=1, color="lightseagreen", size=1.5)+
  theme_light()+
  labs(title="Observed y vs fitted y in case2")+
  theme(plot.title = element_text(hjust=0.5))
# pseudo R^2
1 - sum((test_y - pred_y2)**2) / sum((test_y - mean(test_y))^2)
# feature importance plot
data.frame(xgb.importance(colnames(dtrain), model = xgb2)) %>%
  ggplot(aes(reorder(Feature, Frequency), Frequency))+
  geom_bar(stat="identity", fill="lightcoral")+
  coord_flip()+theme_light()+
```

```
  labs(x="Feature", title="Feature importance for case2")+
  theme(plot.title = element_text(hjust=0.5))

#### Modeling 3: GridSearch
tune_grid <- expand.grid(max_depth = c(4, 6, 8),
                         eta = c(0.01,0.1),
                         colsample_bytree = c(0.6, 0.8),
                         gamma = c(0.5,0.75,1),
                         nrounds = c(100,200),
                         subsample = 1,
                         min_child_weight = 1)
start_time <- Sys.time()
xgb_trcontrol = trainControl(
  method = "cv",
  number = 3,
  allowParallel = TRUE,
  verboseIter = FALSE,
  returnData = FALSE
)

set.seed(777)
xgb_model = train(
  train_x, train_y,
  trControl = xgb_trcontrol,
  tuneGrid = tune_grid,
  method = "xgbTree",
  verbosity = 0
```

```
)
end_time <- Sys.time()
cat("실행시간: ", end_time-start_time)


xgb_model$bestTune
# best parameter  사용
param <- list(objective              = "reg:squarederror",
              eval_metric            = "rmse",
              max_depth              = 8,
              eta                    = 0.1,
              gamma                  = 1,
              colsample_bytree = 0.8)


start_time <- Sys.time()
xgb3 <- xgb.train(params    = param,
                  data       = dtrain,
                  nrounds = 200)
end_time <- Sys.time()
cat("실행시간: ", end_time-start_time)
pred_y3 <- predict(xgb3, dtest)
rmse3 <- sqrt(mean((test_y - pred_y3)^2))
rmse3
# observed y vs fitted y
y_df <- data.frame(obsy = test_y, predy=pred_y3)
y_df %>% ggplot(aes(obsy, predy)) +
   geom_point(colour="lightcoral")+
   geom_abline(intercept = 0, slope=1, color="lightseagreen", size=1.5)+
```

```
  theme_light()+
  labs(title="Observed y vs fitted y in case3")+
  theme(plot.title = element_text(hjust=0.5))
# pseudo R^2
1 - sum((test_y - pred_y3)**2) / sum((test_y - mean(test_y))^2)
# feature importance plot
data.frame(xgb.importance(colnames(dtrain), model = xgb3)) %>%
  ggplot(aes(reorder(Feature, Frequency), Frequency))+
  geom_bar(stat="identity", fill="lightcoral")+
  coord_flip()+theme_light()+
  labs(x="Feature", title="Feature importance for case3")+
  theme(plot.title = element_text(hjust=0.5))

## CatBoost
train_pool <- catboost.load_pool(
  data = train_x,
  label = train_y
)

test_pool <- catboost.load_pool(
  data = test_x,
  label = test_y
)

#### Modeling 1: catboost default 파라미터 사용
##### 1-1. Plain
params <- list(
```

```
  loss_function = "RMSE",
  verbose=100
)
start_time <- Sys.time()
cat1_1 <- catboost.train(learn_pool=train_pool, test_pool=test_pool, params=params)
end_time <- Sys.time()
cat("실행시간: ", end_time-start_time)
pred_y11 <- catboost.predict(cat1_1, test_pool)
rmse11 <- sqrt(mean((test_y - pred_y11)^2))
rmse11
# observed y vs fitted y
y_df <- data.frame(obsy = test_y, predy=pred_y11)
y_df %>% ggplot(aes(obsy, predy)) +
  geom_point(colour="lightcoral")+
  geom_abline(intercept = 0, slope=1, color="lightseagreen", size=1.5)+
  theme_light()+
  labs(title="Observed y vs fitted y in case1-1")+
  theme(plot.title = element_text(hjust=0.5))
# pseudo R^2
1 - sum((test_y - pred_y11)**2) / sum((test_y - mean(test_y))^2)
imp1 = catboost.get_feature_importance(cat1_1,
                                       pool = NULL,
                                       type = 'FeatureImportance',
                                       thread_count = -1)
impdf <- rownames_to_column(data.frame(imp1))
impdf %>% ggplot(aes(reorder(rowname, imp1), imp1))+
  geom_bar(stat='identity', fill="lightcoral")+
```

```
  labs(x='feature', y='importance', title="Feature importance plot for model1-1")+ coord_flip()+
  theme_light()+theme(plot.title = element_text(hjust=0.5))


##### 1-2. Ordered
params <- list(
  boosting_type="Ordered",
  loss_function = "RMSE",
  verbose=100
)
start_time <- Sys.time()
cat1_2 <- catboost.train(learn_pool=train_pool, test_pool=test_pool, params=params)
end_time <- Sys.time()
cat("실행시간: ", end_time-start_time)
pred_y12 <- catboost.predict(cat1_2, test_pool)
rmse12 <- sqrt(mean((test_y - pred_y12)^2))
rmse12
# observed y vs fitted y
y_df <- data.frame(obsy = test_y, predy=pred_y12)
y_df %>% ggplot(aes(obsy, predy)) +
  geom_point(colour="lightcoral")+
  geom_abline(intercept = 0, slope=1, color="lightseagreen", size=1.5)+
  theme_light()+
  labs(title="Observed y vs fitted y in case1-2")+
  theme(plot.title = element_text(hjust=0.5))
# pseudo R^2
1 - sum((test_y - pred_y12)**2) / sum((test_y - mean(test_y))^2)
imp1 = catboost.get_feature_importance(cat1_2,
```

```
                                        pool = NULL,
                                        type = 'FeatureImportance',
                                        thread_count = -1)
impdf <- rownames_to_column(data.frame(imp1))
impdf %>% ggplot(aes(reorder(rowname, imp1), imp1))+
  geom_bar(stat='identity', fill="lightcoral")+
  labs(x='feature', y='importance', title="Feature importance plot for model1-2")+ coord_flip()+
  theme_light()+theme(plot.title = element_text(hjust=0.5))

#### Modeling 2:  파라미터  조정
##### 2-1. Plain
params <- list(
  iterations=1000,
  depth=2,
  learning_rate=0.05,
  random_seed = 777,
  loss_function = "RMSE",
  verbose=100
)
start_time <- Sys.time()
cat2_1 <- catboost.train(learn_pool=train_pool, test_pool=test_pool, params=params)
end_time <- Sys.time()
cat("실행시간: ", end_time-start_time)
pred_y21 <- catboost.predict(cat2_1, test_pool)
rmse11 <- sqrt(mean((test_y - pred_y21)^2))
rmse11
# observed y vs fitted y
```

```
y_df <- data.frame(obsy = test_y, predy=pred_y21)
y_df %>% ggplot(aes(obsy, predy)) +
  geom_point(colour="lightcoral")+
  geom_abline(intercept = 0, slope=1, color="lightseagreen", size=1.5)+
  theme_light()+
  labs(title="Observed y vs fitted y in case2-1")+
  theme(plot.title = element_text(hjust=0.5))
# pseudo R^2
1 - sum((test_y - pred_y21)**2) / sum((test_y - mean(test_y))^2)
imp1 = catboost.get_feature_importance(cat2_1,
                                       pool = NULL,
                                       type = 'FeatureImportance',
                                       thread_count = -1)
impdf <- rownames_to_column(data.frame(imp1))
impdf %>% ggplot(aes(reorder(rowname, imp1), imp1))+
  geom_bar(stat='identity', fill="lightcoral")+
  labs(x='feature', y='importance', title="Feature importance plot for model2-1")+ coord_flip()+
  theme_light()+theme(plot.title = element_text(hjust=0.5))

##### 2-2. Ordered
params <- list(
  boosting_type="Ordered",
  iterations=1000,
  depth=2,
  learning_rate=0.05,
  random_seed = 777,
  loss_function = "RMSE",
```

```
  verbose=100
)
start_time <- Sys.time()
cat2_2 <- catboost.train(learn_pool=train_pool, test_pool=test_pool, params=params)
end_time <- Sys.time()
cat("실행시간: ", end_time-start_time)
pred_y22 <- catboost.predict(cat2_2, test_pool)
rmse12 <- sqrt(mean((test_y - pred_y22)^2))
rmse12
# observed y vs fitted y
y_df <- data.frame(obsy = test_y, predy=pred_y22)
y_df %>% ggplot(aes(obsy, predy)) +
  geom_point(colour="lightcoral")+
  geom_abline(intercept = 0, slope=1, color="lightseagreen", size=1.5)+
  theme_light()+
  labs(title="Observed y vs fitted y in case2-2")+
  theme(plot.title = element_text(hjust=0.5))
# pseudo R^2
1 - sum((test_y - pred_y22)**2) / sum((test_y - mean(test_y))^2)
imp1 = catboost.get_feature_importance(cat2_2,
                                       pool = NULL,
                                       type = 'FeatureImportance',
                                       thread_count = -1)
impdf <- rownames_to_column(data.frame(imp1))
impdf %>% ggplot(aes(reorder(rowname, imp1), imp1))+
  geom_bar(stat='identity', fill="lightcoral")+
  labs(x='feature', y='importance', title="Feature importance plot for model2-1")+ coord_flip()+
```

```
  theme_light()+theme(plot.title = element_text(hjust=0.5))


#### Modeling 3. Gridsearch

##### 3-1. Plain

start_time <- Sys.time()


fit_control <- trainControl(method = "cv",

                            number = 3)


grid <- expand.grid(depth = c(4, 6, 8),

                    learning_rate = c(0.01, 0.1),

                    iterations = c(100, 200),

                    l2_leaf_reg = c(0.1, 1, 3),

                    rsm = c(0.5, 0.85),

                    border_count = c(150, 254))


report <- train(train_x, train_y,

                method = catboost.caret,

                logging_level = 'Silent',

                preProc = NULL,

                tuneGrid = grid, trControl = fit_control)


end_time <- Sys.time()

cat("실행시간: ", end_time-start_time)

print(report)


# best parameter로  모델  적합
```

```
params <- list(
  iterations=200,
  depth=8,
  learning_rate=0.1,
  l2_leaf_reg = 0.1,
  rsm=0.85,
  border_count=254,
  random_seed = 777,
  loss_function = "RMSE",
  verbose=100
)
start_time <- Sys.time()
cat3_1 <- catboost.train(learn_pool=train_pool, test_pool=test_pool, params=params)
end_time <- Sys.time()
cat("실행시간: ", end_time-start_time)
pred_y31 <- catboost.predict(cat3_1, test_pool)
rmse31 <- sqrt(mean((test_y - pred_y31)^2))
rmse31
# observed y vs fitted y
y_df <- data.frame(obsy = test_y, predy=pred_y31)
y_df %>% ggplot(aes(obsy, predy)) +
  geom_point(colour="lightcoral")+
  geom_abline(intercept = 0, slope=1, color="lightseagreen", size=1.5)+
  theme_light()+
  labs(title="Observed y vs fitted y in case3-1")+
  theme(plot.title = element_text(hjust=0.5))
# pseudo R^2
```

```
1 - sum((test_y - pred_y31)**2) / sum((test_y - mean(test_y))^2)

imp1 = catboost.get_feature_importance(cat3_1,

                                       pool = NULL,

                                       type = 'FeatureImportance',

                                       thread_count = -1)

impdf <- rownames_to_column(data.frame(imp1))

impdf %>% ggplot(aes(reorder(rowname, imp1), imp1))+

  geom_bar(stat='identity', fill="lightcoral")+

  labs(x='feature', y='importance', title="Feature importance plot for model3-1")+ coord_flip()+

  theme_light()+theme(plot.title = element_text(hjust=0.5))


##### 3-2. Ordered

start_time <- Sys.time()


fit_control <- trainControl(method = "cv",

                            number = 3)


grid <- expand.grid(depth = c(4, 6, 8),

                    learning_rate = c(0.01, 0.1),

                    iterations = c(100, 200),

                    l2_leaf_reg = c(0.1, 1, 3),

                    rsm = c(0.5, 0.85),

                    border_count = c(150, 254))


report <- train(train_x, train_y,

                method = catboost.caret,

                logging_level = 'Silent',
```

```
                    preProc = NULL,
                    tuneGrid = grid, trControl = fit_control,
                    boosting_type="Ordered")

end_time <- Sys.time()
cat("실행시간: ", end_time-start_time)
print(report)
# best parameter  사용
params <- list(
  boosting_type="Ordered",
  iterations=200,
  depth=8,
  learning_rate=0.1,
  l2_leaf_reg = 0.1,
  rsm=0.85,
  border_count=150,
  random_seed = 777,
  loss_function = "RMSE",
  verbose=100
)
start_time <- Sys.time()
cat3_2 <- catboost.train(learn_pool=train_pool, test_pool=test_pool, params=params)
end_time <- Sys.time()
cat("실행시간: ", end_time-start_time)
pred_y32 <- catboost.predict(cat3_2, test_pool)
rmse32 <- sqrt(mean((test_y - pred_y32)^2))
rmse32
```

```
# observed y vs fitted y
y_df <- data.frame(obsy = test_y, predy=pred_y32)
y_df %>% ggplot(aes(obsy, predy)) +
  geom_point(colour="lightcoral")+
  geom_abline(intercept = 0, slope=1, color="lightseagreen", size=1.5)+
  theme_light()+
  labs(title="Observed y vs fitted y in case3-2")+
  theme(plot.title = element_text(hjust=0.5))
# pseudo R^2
1 - sum((test_y - pred_y32)**2) / sum((test_y - mean(test_y))^2)
imp1 = catboost.get_feature_importance(cat3_2,
                                       pool = NULL,
                                       type = 'FeatureImportance',
                                       thread_count = -1)
impdf <- rownames_to_column(data.frame(imp1))
impdf %>% ggplot(aes(reorder(rowname, imp1), imp1))+
  geom_bar(stat='identity', fill="lightcoral")+
  labs(x='feature', y='importance', title="Feature importance plot for model3-2")+ coord_flip()+
  theme_light()+theme(plot.title = element_text(hjust=0.5))
```